Machine Learning & Deep Learning

UniVR - Dipartimento di Informatica

Tommaso Toffali

1° Semestre 2026/2027

Indice

I Introduction to Machine Learning **8**

1 What is Machine Learning? **8**
1.1 Machine Learning vs Statistical Modelling 8
1.2 Conventional Programming vs Machine Learning 8
1.2.1 Conventional programming 8
1.2.2 Machine Learning 8
1.3 The Machine Learning Recipe 9
1.3.1 The Key ingredients 9

2 Machine Learning Tasks **9**
2.1 Classification task 9
2.2 Regression task 10
2.3 Density Estimation task 10
2.4 Clustering task 10
2.5 Dimensionality Reduction Task 10

3 Data in Machine Learning **10**
3.1 Data representation 10
3.2 Features 11

4 Data Distributions and Learning Types **11**
4.1 Supervised learning 11
4.1.1 Classification and Regression 11
4.2 Unsupervised Learning 11
4.2.1 Density Estimation, Clustering and Dimensionality reduction 11
4.3 Summary of Learning Types 12

5 Data sampling and dataset splitting **12**
5.1 The data distribution problem 12
5.2 Training, Validation and Test sets 12
5.3 How to use the three sets 13
5.3.1 Learning/Training Phase 13
5.3.2 Testing/Model evaluation Phase 13
5.4 The importance of Distribution similarity 13
5.4.1 Correct Scenario: Same Distribution 13
5.4.2 Incorrect Scenario: Different Distribution 13

6 Training and Testing: The role of features **14**
6.1 Training (Learning, Induction) 14
6.2 Testing 14
6.3 Learning and Generalization 15

7 The complete Machine Learning Recipe **15**
7.1 The five ingredients of Machine Learning 15
7.2 The interplay of Ingredients 16

8 Model and Hypothesis Space **16**
8.1 What is a Model? . . . 16
8.2 Types of Learning Paradigms . . . 16
8.2.1 Supervised Learning . . . 17
8.2.2 Unsupervised Learning . . . 17
8.2.3 Semi-Supervised Learning . . . 17

9 Model-Based vs. Instance-Based Learning **17**
9.1 Model-Based (Parametric Learning) . . . 18
9.2 Instance-Based (Nonparametric) Learning . . . 18

10 Error Function (Objective Function) **19**
10.1 What is an Error function? . . . 19
10.2 Types of error functions . . . 19

11 Example: Polynomial Fitting **20**
11.1 Problem setup . . . 20
11.2 Model: Polynomial Functions . . . 20
11.3 Error function: Mean Squared Error . . . 21

12 Underfitting and Overfitting **23**
12.1 Definitions . . . 23
12.2 The Bias-Variance Trade-off . . . 23
12.3 Diagnostic Matrix . . . 24
12.4 How to handle onverfitting . . . 24
12.5 How to handle unverfitting . . . 24
12.6 Polynomial Fitting Example Revisited . . . 25

13 Regularization **25**
13.1 What is Regularization? . . . 26
13.2 The regularized objective function . . . 26
13.3 Effect of Regularization . . . 27
13.4 Regularization in Polynomial Fitting . . . 27
13.4.1 Effect of the Regularization Parameter λ . . . 27
13.4.2 Regularization Path . . . 27
13.5 Generalization and Data Size . . . 28

II Model Evaluation and Regression Methods 29

14 Model evaluation for Classification and Regression **29**
14.1 Recap: Train, Validation, and Test Sets . . . 29
14.1.1 The three dataset . . . 29
14.2 The importance of the validation set . . . 30
14.2.1 Why separate the validation set? . . . 30
14.3 Specific Rules for Dataset Splitting . . . 31
14.3.1 Priority 1: Accurate Error Estimation . . . 31
14.3.2 Priority 2: Accurate Classifier Learning . . . 31
14.3.3 The critical Rule . . . 32

15 Cross Validation **32**
15.1 When do we need Cross Validation? . . . 32
15.2 Key characteristics of Cross Validation . . . 32
15.3 Fundamental Rules of Cross Validation . . . 33
15.4 K-Fold Cross Validation . . . 33
15.4.1 How K-Fold Cross Validation works . . . 33
15.4.2 Visual Representation of K-Fold Cross Validation . . . 34
15.5 Example: 5-Fold Cross Validation . . . 34
15.5.1 The process in Detail . . . 35
15.6 Advantages of Cross Validation . . . 35
15.7 Disadvantages of Cross Validation . . . 35

16 Leave-One-Out Cross Validation **36**
16.1 What is Leave-One-Out Cross Validation? . . . 36
16.2 How LOOCV Works . . . 36
16.3 Pros and Cons of LOOCV . . . 37
16.3.1 Advantages . . . 37
16.3.2 Disadvantages . . . 37

17 Stratification in Cross Validation **37**
17.1 The problem: Imbalanced Classes . . . 37
17.2 What is Stratification? . . . 37
17.3 How Stratified K-Fold Works . . . 38
17.4 Example: Stratified 5-Fold Cross Validation . . . 38
17.4.1 Imbalanced Dataset . . . 38
17.4.2 Split the Dataset into Five Folds with Stratification . . . 38
17.4.3 Train and Validate . . . 39
17.4.4 Average the performance Metrics . . . 39
17.5 Why Stratification Matters . . . 40

18 Evaluation Measures **40**
18.1 Why do we need Evaluation Measures? . . . 40
18.2 Evaluation Measures by Task Type . . . 40
18.2.1 Classification . . . 41
18.2.2 Regression . . . 41
18.2.3 Clustering . . . 41

19 Evaluation Measures of Classification **41**
19.1 Types of Classification Problems . . . 41
19.2 The confusion matrix . . . 41
19.2.1 Confusion Matrix for Binary classification . . . 42
19.2.2 Understanding the Confusion Matrix . . . 42
19.3 Classification Error and Accuracy . . . 43
19.3.1 Classification error . . . 43
19.3.2 Accuracy . . . 43
19.4 The problem with Accuracy: Imbalanced Classes . . . 43
19.4.1 Example: The imablanced Dataset Problem . . . 43
19.5 Misses and False Alarms . . . 44
19.5.1 False Alarm Rate (False Positive Rate) . . . 44
19.5.2 Miss Rate (False Negative Rate) . . . 44

19.5.3 Recall (True Positive Rate) . . . 44
19.5.4 Precision . . . 44
19.6 Cost of the Task . . . 45
19.6.1 The need for a single evaluation measure . . . 45
19.6.2 weighting Errors by Cost . . . 45
19.7 F-Measure (F1 Score) . . . 46
19.7.1 Properties of F1 Score . . . 46
19.7.2 Generalized F-Measure . . . 46
19.8 Multiclass Classification - Evaluation . . . 46
19.8.1 Multiclass confusion matrix . . . 47
19.8.2 Multiclass Metrics . . . 47
19.8.3 Macro vs. Micro averaging . . . 47
19.9 ROC Curve - Receiver Operating Characteristic . . . 47
19.9.1 What is the ROC curve? . . . 48
19.9.2 Uses of ROC Curves . . . 48
19.9.3 Understanding the ROC space . . . 48
19.9.4 Area under the ROC Curve (AUC) . . . 48
19.9.5 Interpreting AUC Values . . . 48
19.9.6 Advantages of ROC and AUC . . . 49

20 Regression Evaluation Metrics 49
20.1 The Fundamental Difference . . . 49
20.2 Mean Squared Error (MSE) . . . 49
20.2.1 Properties of MSE . . . 50
20.3 Mean absolute error (MAE) . . . 50
20.3.1 Properties of MAE . . . 50
20.4 Choosing Between MSE and MAE . . . 51
20.5 Other Regression Metrics . . . 51
20.5.1 Root Mean Squared Error (RMSE) . . . 51
20.5.2 R-Squared (R^2) Score . . . 51

III Bayes Classifier and Nonparametric Classifiers 52

21 Bayes Classifier 52
21.1 Introduction . . . 52
21.1.1 Goal of Bayes Classifier . . . 52
21.2 An Easy Example . . . 52
21.2.1 Decsion Rule . . . 53
21.3 Another example: Adding Measurements . . . 53
21.3.1 Visual interpretation . . . 53
21.4 Bayes Formula . . . 53
21.4.1 Understanding the components . . . 54
21.4.2 Bayes Decision Rule . . . 54
21.5 Bayes Decision Rule: Detailed Analysis . . . 55
21.5.1 Understanding the Posterior . . . 55
21.5.2 The Decision Rule . . . 55

22 Naïve Bayes Classifier **55**
22.1 Core Principles . . . 56
22.2 The Naïve Independence Assumption . . . 56
22.2.1 Training the Classifier . . . 56
22.3 Example: Email Spam Classification . . . 56
22.3.1 Training Data . . . 57
22.3.2 Step 1: Compute Class Priors . . . 57
22.3.3 Step 2: Estimate Conditional Probabilities . . . 57
22.3.4 Step 3: Classify a New Email . . . 58
22.4 The zero probability Problem . . . 58
22.4.1 Solution: Laplace Smoothing . . . 58
22.5 Advantages and disadvantages . . . 59
22.6 Classification Rule Summary . . . 59
22.6.1 Key Points . . . 59
22.7 Problems with Bayesian Classifier . . . 59
22.7.1 What we actually have . . . 60
22.8 Challenges in Estimating Conditional Densities . . . 60
22.9 Solution: Parametric Models . . . 60
22.9.1 Maximum Likelihood Estimation (MLE) . . . 61
22.9.2 Gaussian Naïve Bayes . . . 61
22.10Summary: From Theory to Practice . . . 62

23 MLE in Naïve Bayes: Practical Implementation **62**
23.1 MLE for Discrete (Categorical) Features . . . 62
23.1.1 (a) Prior via MLE . . . 63
23.1.2 (b) Likelihood via MLE . . . 63
23.2 MLE for Continuos Features: Gaussian Model . . . 63
23.3 Complete Example: Fruit Classification . . . 64
23.3.1 Step 1: Estimate Gaussian Parameters (MLE) . . . 64
23.3.2 Step 2: Compute the Likelihood for a New Sample . . . 65
23.3.3 Step 3: Multiply by Class Prior . . . 65

24 Nonparametric Classifiers **66**
24.0.1 Introduction to Nonparametric Methods . . . 66

25 Histogram-Based Density Estimation **66**
25.0.1 How Histogram Work . . . 67
25.0.2 Hisogram Parameters . . . 67
25.1 Drawbacks of Histograms . . . 67
25.2 The curse of Dimensionality . . . 68
25.2.1 Why it matters . . . 68
25.2.2 Mitigatin the Curse of Dimensionality . . . 69
25.3 Probability Density: Mathematical Foundation . . . 69
25.3.1 Key concepts . . . 69
25.4 Binomial Distribution Approach . . . 70
25.4.1 Setup . . . 70
25.4.2 Mean and Variance of the Ratio . . . 71
25.5 Density Estimation Formula . . . 71
25.5.1 Small Region Assumption . . . 71
25.5.2 Combining the Results . . . 71

25.5.3 Combining the Results . . . 71
25.6 The Trade-off Problem . . . 72
25.6.1 The dilemma . . . 72
25.7 General Nonparametric Density Estimation Formula . . . 72
25.8 Two Approaches to Nonparametric Density Estimation . . . 73
25.8.1 Comparison of Approaches . . . 73
25.8.2 Choosing Parameters . . . 73

26 k-Nearest Neighbors (k-NN) 74
26.1 Visual Comparison: k-NN vs. Parzen Windows . . . 74
26.2 Parzen Window: Practical Consideration . . . 74
26.3 k-Nearest Neighbors Algorithm . . . 74
26.3.1 Basic principle . . . 74
26.4 k-NN for Classification . . . 75
26.4.1 1-Nearest Neighbor (1-NN) . . . 75
26.4.2 Generalizing to k-NN . . . 76
26.4.3 Majority Voting . . . 76
26.5 Advantages and Disadvantages of k-NN . . . 77
26.6 Practical Considerations . . . 77
26.6.1 Feature Scaling . . . 77
26.6.2 Computational Complexity . . . 77
26.7 Distance Metrics in k-NN . . . 77
26.8 Weighted k-NN . . . 78
26.8.1 The problem with Equal Weights . . . 78
26.8.2 Solution: Distance-Based Weighting . . . 78
26.9 Importance of Feature Scaling (Revisited) . . . 79
26.9.1 Why k is Sensitive . . . 79
26.10 Mahalanobis Distance . . . 79
26.10.1 When Euclidean Distance is not Enought . . . 79
26.10.2 Definition . . . 79
26.10.3 When to Use Mahalanobis distance . . . 80
26.10.4 Visual Example: Euclidean vs. Mahalanobis Distance . . . 81
26.11 How to select k in k-NN . . . 82
26.11.1 Common Heuristics . . . 82
26.11.2 Handling Ties (Undetermined Cases) . . . 84
26.12 k-NN: Pros and Cons . . . 84
26.12.1 Advantages (Pros) . . . 84
26.12.2 Disadvantages (Cons) . . . 85
26.13 How k-NN Was Born: PDF Estimation . . . 86
26.13.1 Going back to PDF Estimation . . . 86
26.13.2 Density Estimation and Nearest-Neighbor Rule . . . 87

IV Decision Trees and Ensemble Methods 88

27 Introduction to Decision Trees 88
27.1 Decision Tree Structure . . . 88
27.2 How decision Trees Work . . . 88
27.3 Decision Tree Inference: Non-terminal Nodes . . . 89
27.4 Decision Tree Inference: Leaf Nodes . . . 89

27.5 Formal Decision Tree Function 89
27.6 Decision Tree Inference Example 89

V Linear Discriminant Functions and Classiification Methods 90

28 Linear Discriminant functions 90
28.1 Larning the Linear Discriminant Function 91
28.2 Linear Separability 92
28.3 Properties of Linear Discriminant Functions 92

29 Linear Classifiers 93
29.1 Decision Boundary 93
29.2 The Role of Weights 94
29.3 Which Hyperplanes? 94
29.4 Optimization Approaches 95
29.4.1 Unit-Length Weight Vector 95
29.4.2 Minimum-Length Weight Vector 95
29.5 The Margin 95

30 Multi-Class Problem 96

31 LDF: Augmented Feature Vector 96

32 LDF: Training Error 97
32.1 Criterion for Determining Weights 97

33 Perceptron 97
33.1 Perceptron Criterion Function 97
33.2 Convergence Limitation 97
33.3 Perceptron Gradient Descent 98
33.4 Perceptron Example (Non-Separable Scenario) 98
33.5 Applying the Perceptron Method 99
33.6 Need for Relaxation 100
33.7 Perceptron Summary 100

Parte I
Introduction to Machine Learning

1 What is Machine Learning?

Machine Learning is a branch of Artificial Intelligence that enables software to use data to find solutions to specific tasks without being explicitly programmed to do so.

1.1 Machine Learning vs Statistical Modelling

In traditional statistical modelling, we follow a structured process:

1. Collect data
2. Verify and clean the data (correct or discard if not clean)
3. Use the clean data to test hypotheses, make predictions and forecasts

In contrast, Machine Learning takes a different approach:

- It is the **data** that determines which analytic techniques to use
- The computer uses data to **train** algorithms to find patterns and make predictions
- Algorithms are no longer **static** but become **dynamic**

1.2 Conventional Programming vs Machine Learning

The fundamental difference between conventional programming and machine learning can be undersood through their inputs and outputs:

1.2.1 Conventional programming

- **Input**: Program + Data
- **Output**: Result
- You write the program, give it data, and get results

1.2.2 Machine Learning

- **Input**: Data + Result
- **Output**: Result
- You give the computer data and desired results, and it learns to generate a program/algorithm

1.3 The Machine Learning Recipe

Definizione 1.1 (Fundamental Steps of ML)

Collect data → Define models → Define objective function → Minimize error

The ML recipe consists of 4 fundamental step:

1. Collect data
2. Define a family of possible models
3. Define an objective (error) function to quantify how well a model fits the data
4. Find the model that minimize the error function (training/learning a model)

1.3.1 The Key ingredients

Esempio 1.2 (Essential Ingredient)

Task • Data • Model • Objective Function • Learning Algorithm

Every machine learning problem requires five essential ingredients:

- **Task**: The problem we want to solve
- **Data**: The information we use to learn
- **Model hypothesis**: The family of functions we consider
- **Objective function**: How we measure success
- **Learning algorithm**: How we fine the best model

2 Machine Learning Tasks

A task represent the type of prediction being made to solve a problem on some data. We can identify a task with the set of functions that can potentially solve the problem. In generale, it consists of functions assigning each **input** $x \in \mathcal{X}$ an **output** $y \in \mathcal{Y}$.

2.1 Classification task

Definizione 2.1 (Classification)

Find a function $f \in \mathcal{Y}^{\mathcal{X}}$ assigning each input $x \in \mathcal{X}$ a **discrete** label, $f(x) \in \mathcal{Y} = \{c_1, \ldots, c_k\}$.

In classification, the output space is a finite set of discrete categories or classes. The goal is to learn a decision boundary that separates different classes.

2.2 Regression task

Definizione 2.2 (Regression)

Find a function $f \in \mathcal{Y}^{\mathcal{X}}$ assigning each input $x \in \mathcal{X}$ a **continuos** label, $f(x) \in \mathcal{Y} = \mathbb{R}$.

In regression, the output is a continuos value, allowing for predictions of quantities such as prices, temperatures, or probabilities.

2.3 Density Estimation task

Definizione 2.3 (Density Estimation)

Find a probability distribuition $f \in \Delta(\mathcal{X})$ that best fits the data $x \in \mathcal{X}$.

There is no reference to an output space (as in calssification and regression task). Instead, we make a reasoning on the **input** x itself, trying to model the underlying probability distribution that generated the data.

2.4 Clustering task

Definizione 2.4 (Clustering)

Find a function $f \in \mathbb{N}^{\mathcal{X}}$ that assigns each input $x \in \mathcal{X}$ a **cluster index** $f(x) \in \mathbb{N}$.

All points mapped to the same index form a cluster. The goal is to group similar data points together without prior knowledge of the groups.

2.5 Dimensionality Reduction Task

Definizione 2.5 (Dimensionality Reduction)

Find a function $f \in \mathcal{Y}^{\mathcal{X}}$ that **maps** each (**high-dimensional**) input $x \in \mathcal{X}$ to a **lower-dimensional** embedding $f(x) \in \mathcal{Y}$, where $dim(\mathcal{Y}) < dim(\mathcal{X})$.

3 Data in Machine Learning

3.1 Data representation

We represent inputs of out algorithm as $x \in \mathcal{X}$ and output as $y \in \mathcal{Y}$. The dataset is the set of both, where each input is associated with an output.

Key consideration:

- The quality of the data is crucial for the performance of the algorithm
- It is equally important to have a large amount of data to learn effective models
- Poor quality or insufficient data can lead to poor model performance

3.2 Features

Definizione 3.1 (Features)

Features are the measurable properties or charateristics of the phenomenon being observed

Examples:

- To identify a flower: color, petal length, petal width, sepal length, sepal width
- To identify a fruit: color, shape, size, texture, weight
- To classify images: pixel values, edges, textures, shapes

We can say that features are how algorithm "sees" the data and are in general represented with (fixed-size) vectors.

4 Data Distributions and Learning Types

In machine learning, the information about the problem we want to solve is represented in the form of a data distribution, usually denoted as p_{data}.

4.1 Supervised learning

4.1.1 Classification and Regression

The data distribution is over pairs of inputs and outputs:

$$p_{data} \in \Delta(\mathcal{X} \times \mathcal{Y})$$

Here:

- $\mathcal{X}$ is the input space
- $\mathcal{Y}$ is the output space (labels or targets)
- The goal is to learn a mapping from inputs to outputs using labeled data

4.2 Unsupervised Learning

4.2.1 Density Estimation, Clustering and Dimensionality reduction

The data distribution is only over the input space:

$$p_{data} \in \Delta(\mathcal{X})$$

Here:

- We do not have explicit labels or targets
- The goal is to discover structure, patterns or representations within the data itself

4.3 Summary of Learning Types

Task type	Data distribution	Learning Type
Classification, Regression	$p_{data} \in \Delta(\mathcal{X} \times \mathcal{Y})$	Supervised Learning
Density Estimation, Clustering, Dimensionality Reduction	$p_{data} \in \Delta(\mathcal{X})$	Unsupervised Learning

Note. In **supervised learning**, each data point consistes of an input and corresponding output (label). In **unsupervised learning**, each data point consists only of the input, and the algorithm tries to find pattersn or structure without explicit labels.

The distinction is fundamental in machine learning, as it determines the type of algorithms and approaches used to solve different problems.

5 Data sampling and dataset splitting

5.1 The data distribution problem

In both supervised and unsupervised learning, the true data distribution p_{data} is typically **unknown**. This presents a fundamental challenge:

- We do not have direct access to the underlying probability distribution that generates data
- We only have access to a finite sample of data points drawn from this distribution

Solution: We perform **sampling** from the distribution. By collecting a representative sample of data, we can approximate the true distribution and use it to train our models.

5.2 Training, Validation and Test sets

To properly evaluate machine learning models, we split out data into three distinct sets. Each set serves a specific purpose inthe machine learning pipeline.

Definizione 5.1 (Dataset Splitting)

A dataset is typically divided into three subsets:

- **Training set** (D_{train}): Used to train the model
- **Validation set** (D_{val}): Used to tune hyperparameters and select the best model
- **Test set** (D_{test}): Used to evaluate the final model performance

Definizione 5.2 (Critical Requirement)

All three sets (training, validation, test) must be sampled from the **same probaility distribution**

5.3 How to use the three sets

The three datasets serve different purposes in the machine learning workflow:

5.3.1 Learning/Training Phase

During the learning pahse, the **training set** is used to train the machine learning algorithm, which learns patterns and realtionships in the data to produce a **program** (model)

The **validation set** plays a crucial role in this phase:

- Evaluates different model configurations
- Tunes hyperparameters
- Selects the best performing model

This process involves iterative feedback between training and validation until we find the optimal configuration.

5.3.2 Testing/Model evaluation Phase

Once the best model is selected, we use the **test set** to evaluate its performance. The trained model is applied to the test data, which it has never seen before. This gives us an unbiased estimated of how well the model will perform in real-world scenarios, telling us whether out model can truly generalize to new data.

5.4 The importance of Distribution similarity

5.4.1 Correct Scenario: Same Distribution

When training, validation, and test sets are sampled from the **same distribution**:

- The sets are **similar** in terms of statistical properties
- They represents the same underlying phenomenon
- However, they **do not overlap** (no data point appears in multiple sets)

This ensures that the model cna generalize well from training to test data.

Esempio 5.3 (Fruits)

If we're classifying fruits, all three sets might contain apples and bananas in similar proportions, but with different specific instances.

5.4.2 Incorrect Scenario: Different Distribution

When training/validation and test come from **different distributions**, the model learns patterns from one distribution but is tested on another. This problematic scenario typically leads to:

- Poor performance on the test set

- Unrealiable predictions on new data
- Inability to generalize to real-world applications

Esempio 5.4 (Different distributions)

Training on apples and bananas, but testing on oranges and limes. The model cannot calssify fruits it has never seen during training

Teorema 5.5 (Key principle)

Training and test data must come from the **same distribution**, violating thi leads to poor generalization

6 Training and Testing: The role of features

6.1 Training (Learning, Induction)

During the training phase, the model learns to distinguish between different classes based on the features provided in the training data.

Esempio 6.1 (Fruit classification)

Consider a fruit classification task with the following training data:

- *red, round, leaf, 3oz, ...* $\rightarrow$ apple
- *green, round, no leaf, 4oz, ...* $\rightarrow$ apple
- *yellow, curved, no leaf, 8oz, ...* $\rightarrow$ banana
- *green, curved, no leaf, 7oz, ...* $\rightarrow$ banana

The model learns to associate patterns in the features (color, shape, presence of leaf, weight) with the corresponding labels (apple or banana). **What distinguishes apples and bananas is based on the features!** The model identifies which feature combinations are characteristic of each class.

6.2 Testing

Once trained, the model can classify new examples based on the same features it learned during training. When presented with a new example described byt the same semantic features, the model make a prediction

Esempio 6.2 (Fruit)

Given a new fruit with features *red, round, no leaf, 4oz, ...*, the model uses the learned patterns to predict whether it is an apple or banana.

Teorema 6.3 (Critical Assumption)

THe features of training and testing data **must be the same!**. The model can only make predictions based on features it was trained on

6.3 Learning and Generalization

Learning is fundamentally about **generalizing** from the training data. The goal is not simply to memorize the training examples, but to learn patterns that apply to new, unseen data.

However, the failure of a machine learning algorithm is foten caused by a **bad selection of training samples**. The key issue is that we might introduce **unwanted correlations** from which the algorithm derives **wrong conclusions**.

Esempio 6.4 (Recognize object in images)

Consider a model trained to recognize objects in images. If all training images were captured on sunny days, the model might learn to associate sunny weather conditions with the objects. When tested on images taken on cloudy days, the model might fail to recognize the same objects because it never encountered cloudy cnditions during training. The model incorrectly learned that sunny weather is a relevant feature for object recognition.

Quality and diversity of training data are crucial. Training data should be representative of all real-world conditions

7 The complete Machine Learning Recipe

We can now revisit the complete machine learning workflow with all its essential ingredients:

7.1 The five ingredients of Machine Learning

Every machine learning problem requires five fundamental components that work toghether:

1. **Task**: The specific problem we want to solve (e.g., classify birds vs. non-birds in images)
2. **Data**: The collection of examples used to train and evaluate the model. Data quality and quantity are critical for success
3. **Model Hypothesis**: The family of function or models we consider as a potential solutions. This defines the space of possible models the learning algorithm can explore.
4. **Objective Function**: A mathematical function that quantifies how well a model performs on the data. The learning algorithm aims to optimize this function
5. **Learning Algorithm**: The method used to search through the model hypothesis space and find the model that optimizes the objective function.

7.2 The interplay of Ingredients

These five ingredients are interconnected:

- The **task** determines what kind of **data** we need and what **model hypothesis** is appropriate
- The **data** influences which features are available and how we define the **objective function**
- The **model hypothesis** constrains what the **learning algorithm** can discover
- The **objective function** guides the **learning algorithm** toward better models

Understanding and carefully selecting each ingredient is essential for building effective machine learning systems.

8 Model and Hypothesis Space

8.1 What is a Model?

Definizione 8.1 (Model)

A **model** is the implementation of a function $f \in \mathcal{F}_{\text{task}}$ that can be tractably computed.

IN practice, when searching for our function f, we don't look at everything in $\mathcal{F}_{\text{task}}$. Instead, we look at a subset called the **hypothesis space**: $\mathcal{H} \subset \mathcal{F}_{\text{task}}$, which is composed of a set of models (e.g., neural networks, decision trees, linear models).

The learning algorithm seeks a **solution within the hypothesis space**. In other words, a model is a simplified representation of the world that we can actually compute and optimize.

Observation. The choice of hypothesis space is crucial:

- If $\mathcal{H}$ is too small, it may not contain a good solution
- If $\mathcal{H}$ is too large, finding the best model becomes computationally intractable
- The hypothesis space defines the types of patterns the model can learn

8.2 Types of Learning Paradigms

Definizione 8.2 (Three Learning Paradigms)

Machine learning can be categorized into different paradigms based on the type and availability of labeled data:

Supervised learning

In supervised learning, we have labeled data where each input is paired with its corresponding output. This includes:

- **Classification**: Predicting discrete labels
- **Regression**: Predicting continuous values

Unsupervised learning

In unsupervised learning, we only have input data without labels. The goal is to discover structure in the data. That includes:

- **Clustering**: Grouping similar data points
- **Density estimation**: Modeling the probability distribution of data
- **Dimensionality Reduction**: Finding lower-dimensional representations

8.2.1 Supervised Learning

In supervised learning, we have labeled data where each input is paried with its corresponding output.

8.2.2 Unsupervised Learning

In unsupervised learning, we only have input data without labels.

8.2.3 Semi-Supervised Learning

Semi-supervised learning addresses scenarios where we have a large amout of unlabeled data and only some labeled examples. This paradigm has gained significant popularity in recent years to leverage large dataset through self-supervised learning techniques.

The key idea is to provide the model with tasks different from the one to be solved, but that allow the model to learn the data distribution from the unlabeled data. Once the model understands the general structure of the data, it can then specialize on the few labeled examples to solve the target task.

Esempio 8.3 (Training a language)

Training a language model on large amounts of unlabeled text to learn general laguage pattern, then fine-tuning it on a small labeled dataset for sentiment analysis.

9 Model-Based vs. Instance-Based Learning

Machine learning algorithms can be further categorized based on how they make predictions:

9.1 Model-Based (Parametric Learning)

Definizione 9.1 (Parametric Model)

A learning model that summarizes data with a **set of parameters of fixed size** (independent of the number of training examples) is called **parametric model**.

Parametric Model Characteristics

Key characteristics:

- The model learns a fixed set of parameters from the training data
- Once trained, the original training data can be discarded
- No matter how much data you give to a parametric model, it won't change its mind about how many parameters it needs.
- The model makes predictions using only the learned parameters

Examples: Naive Bayes, Linear regression, Logistic Regression, Neural Networks.

In a model-based approach, the algorithm learns a function (e.g., decision boundary) described by parameters. For instance, a linear classifier might learn the curve $\alpha x_1 + \beta x_2 + \gamma$ that separates two classes, where α, β, γ are the parameters.

9.2 Instance-Based (Nonparametric) Learning

Definizione 9.2 (Nonparametric Model)

These algorithms **do not learn parameters** but instead use the data itself to compare a new example through similarity metrics.

Nonparametric Model characteristics

Key characteristics:

- The models stores (some or all of) the training data
- Predictions are made by comparing new examples to stored training examples
- The model's complexity can grow with the amount of training data
- No explicit training phase that produces a fixed set of parameters

Example: K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Random Forest.

In an instance-based approach, when a new example arrives, it is compared with nearby training examples and classified according to a similarity policy. For

example, in KNN, a new point is classified based on the majority class of its k nearest neighbors in the training data.

10 Error Function (Objective Function)

To allow an algorithm to learn, we need to provide it with a metric, a measure that helps it understand the direction of the optimal solution we are seeking. For this reason, **performance measures** are defined to enable the algorithm to learn.

Aspect	Model-based	Instance-Based
Parameters	Fixed number of parameters	No fixed parameters
Training data	Can be discarded after training	Must be retained for predictions
Prediction speed	Fast (onlu uses parameters)	Slower (compares with stored data)
Memory	Low (stores only parameters)	High (stores training examples)
Flexibility	Fixed model complexity	Complexity grows with data

10.1 What is an Error function?

In machine learning, **training a model** means finding a **function** which maps a set of values x to a value y. we can calculate how well a predictive model is doing by comparing the **predicted values** with the **true values** for y.

Definizione 10.1 (Training error vs. Test Error)

- If we apply the model to the data it was trained on, we are calculating the **training error**
- If we calculate the error on data which was **unknown** in the training phase, we are calculating the **test error**

The test error is the most important metric because it tells us how well the model generalizes to new, unseen data.

10.2 Types of error functions

There are different types of error functions (also called loss functions or objective functions), each suited for different task:

- **Mean Absolute Error (MAE)**: Measures the average absolute difference between predictions and true values
- **Mean Squared Error (MSE)**: Measures the average squared difference between predictions and true values
- **Binary Cross-Entropy**: Used for binary classification problems
- **Mean Average Precision**: Used for ranking and information retrieval tasks

The choice of error function depends on the task and the desired properties of the model.

11 Example: Polynomial Fitting

Let's consider a concrete example to understand how models, hypothesis spaces, and error functions work toghether.

11.1 Problem setup

Consider regression problem with the following setup:

- **Data**: $\mathcal{D}_n = \{(x_1, y_1), \ldots, (x_n, y_n)\}$.
- Data generated from $sin(2\pi x)+$ noise
- Training set with $n = 10$ points
- **Goal**: Learn a function (shown as the purple line) that fits the data

The correct solution shold capture the underlying sinusoidal pattern while being robust to the noise in the training data.

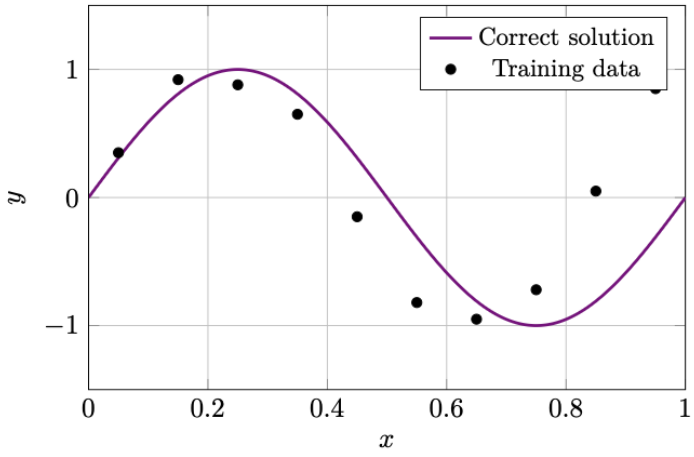


Figura 1: Polynomia Fitting Problem: Data generated from $sin(2\pi x)$ with noise

11.2 Model: Polynomial Functions

Definizione 11.1 (Polynomia Model)

We choose to model the data using polynomia functions:

$$f_w(x) = \sum_{j=0}^{M} w_j x^j$$

where:

- M is the **degree of the polynomial** (a hyperparameter)
- $w = \{w_0, \ldots, w_M\}$ are the **parameters** to be learned
- The **hypotesis space** for a fixed degree M is: $\mathcal{H}_M = \left\{ f_w : w \in \mathbb{R}^{M+1} \right\}$

Different values of M give us different hypothesis spaces:

- $M = 0$: Constant functions (horizontal lines)
- $M = 1$: Linear functions (straight lines)
- $M = 2$: Quadratic functions (parabolas)
- $M = 3$: Cubic functions
- Higher M: More complex curves

11.3 Error function: Mean Squared Error

To measure how well our polynomial fits the data, we use the Mean Squared Error (MSE):

$$E(f; \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^{n} [f(x_i) - y_i]^2$$

This error function:

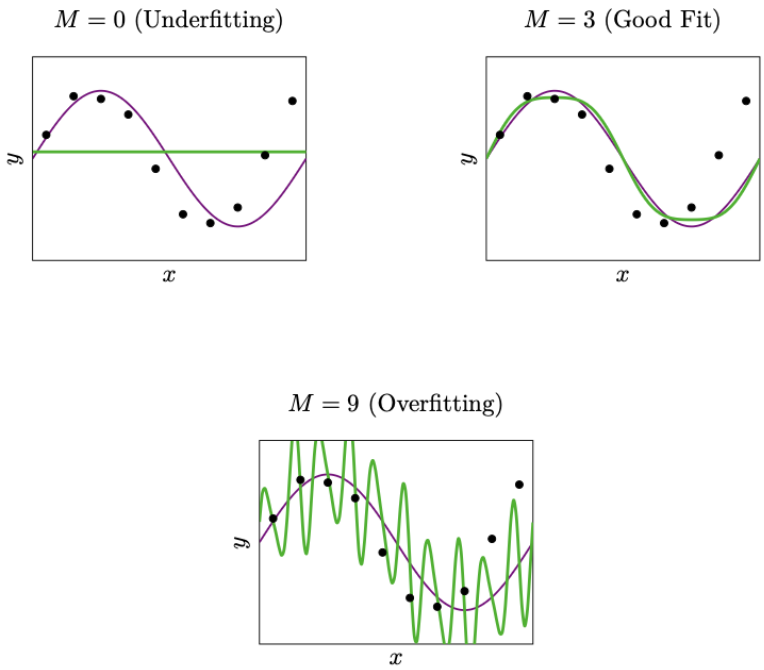


Figura 2: Polynomial fits with different degrees: $M = 0$ (underfitting), $M = 3$ (good fits), $M = 9$ (overfitting). Purple: true function, Green: fitted polynomial, Black dots: training data

- Computes the squared difference between the prediction $f(x_i)$ and the ground-truth label y_i for each point
- Averages these squared differences over all n training points
- Is also called a **pointwise loss** because it measures the error at each individual data point

The learning algorithm's goal is to find the parameters w that minimize this error function:

$$w^* = \arg\min_{w \in \mathcal{R}^{M+1}} E(f_w; \mathcal{D}_n)$$

Choice of Model Comlpexity is Crucial:

- Too small M: Underfitting (cannot captuer pattern)
- Appropriate M: Good fit (captures underlying pattern)
- Too large M: Overfitting (fits noise, poor generalization)

12 Underfitting and Overfitting

Two fundamental problems in ML

Underfitting (too simple) $\leftrightarrow$ Overfitting (too complex)

Two fundamental problems can occur when training machine learning models: underfitting and overfitting. Understanding these concepts is crucial for building models that generalize well.

12.1 Definitions

Definizione 12.1 (Underfitting)

Underfitting is a scenario where a model is unable to capture the relationship between the input (x) and output (y) variables accurately, generating a high error rate on both the training set and unseen data (e.g., testing set).

The model is too simple and has not yet learned from the data. It performs poorly on both training and test data.

Definizione 12.2 (Overfitting)

Overfitting occurs when the model gives accurate predictions for training data (lower training error) but not for testing data (higher testing error).

The model is too sensitive to the training data and has essentially memorized it, including its noise and peculiarities, rather learning the underlying pattern.

12.2 The Bias-Variance Trade-off

The relationship between model complexity, training error can be visualized as follows:

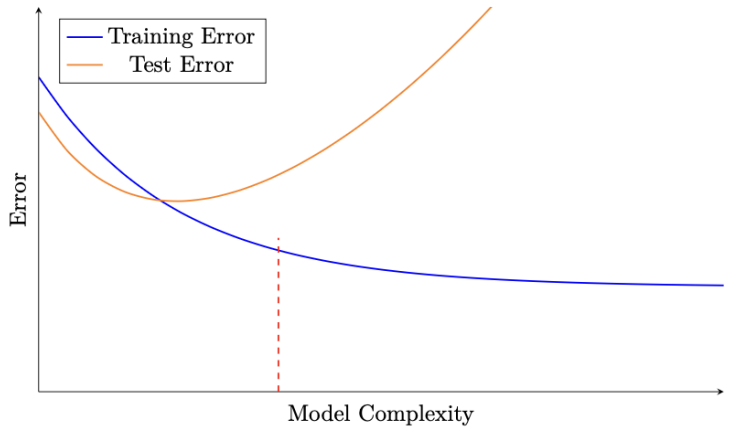


Figura 3: Bias-Variance Trade-off: Training and Test Error vs. Model Complexity

- **Left side of the graph (low complexity)**: High error rate in both training and testing → **Underfitting**
- **Middle Region (optimal complexity)**: Low error rate in both training and testing → **Best Fit**
- **Right side of the graph (high complexity)**: Low error rate in training but high error rate in testing → **Overfitting**

12.3 Diagnostic Matrix

We can dignose the state of our model by examining both training and test errors:

	Low training error	**High training error**
Low Test Error	OK (Goot fit)	Underfitting
High Test Error	Overfitting	Underfitting

12.4 How to handle onverfitting

When the model is too sensitive to training data and overfits, we can apply several strategies:

1. **Try a simpler model**: Use a model with fewer parameteres or lower complexity
2. **Try a less powerful model**: Choose a model architecture with reduced capacity
3. **Increase regularization impact**: Apply techniques that penalize model complexity:
    - Early stopping: stop training before the model overfits
    - L1/L2 regularization: Add penalty terms to the loss function
4. **Use a smaller number of features**:
    - Remove some features that may be causing overfitting
    - Apply feature selection techniques to identify the most relevant features
5. **Get more data**: Increasing the training set size helps the model learn more generalizable patterns

12.5 How to handle unverfitting

When the model has not yet learned from the data and underfits, we can try:

1. **Try more complex model**: Use models with larger number of parameters that can capture more intricate patterns
    - Ensemble learning: Combine multiple models

2. **Less regularization**: Reduce or remove regularization constraints that may be limiting the model's capacity

3. **A larger quantity of features**: Add more features or engineer new features that better capture the underlying relationships (get more features)

12.6 Polynomial Fitting Example Revisited

Let's examine how different polynomial degrees affect the fit:

- $M = 0$ (constant function): Severe underfitting. The horizontal line cannot capture any of the sinusoidal pattern. Both training and validation errors are high
- $M = 1$ (linear function): Still underfitting. A straight line cannot represent the curved pattern. Both errors remain high.
- $M = 3$ (cubic polynomial): Good fit. The model captures the underlying sinusoidal pattern well. Both training and validation errors are low, and the curves are similar
- $M = 9$ (9th-degree polynomial): Overfitting. The model passes through alla training points (very low training error) but oscillates wildly between them. The validation error is high because the model has learned the noise rather than the signal.

The error plot shows:

- **Training error** (blue line): Decresases monotonically as M increases
- **Validation error** (orange line): Decreases initially, reaches a minimum around $M = 3$, then increases again
- The optimal model complexity is where the validation error is minimized

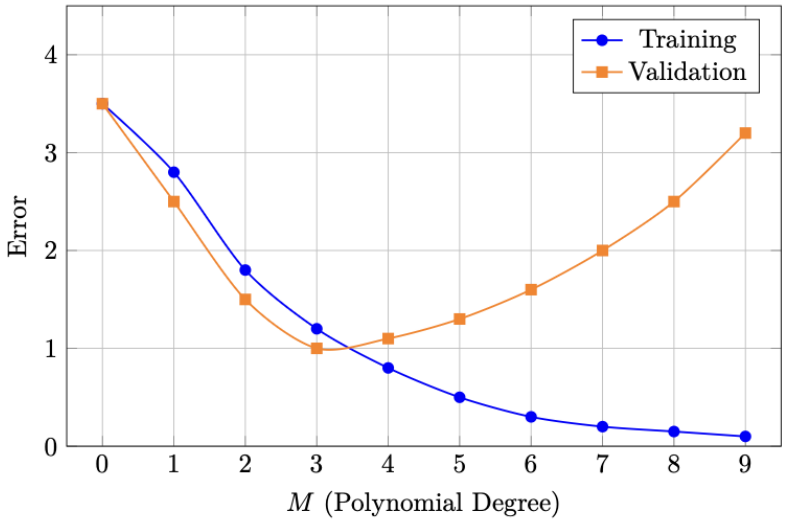
Key Insight

Minimizing training error alone is **not sufficient**
Must monitor validation/test error for generalization

13 Regularization

One of the most important techinques in ML $\rightarrow$ Prevents overfitting and improves generalization

Regularization is one of the most important techniques in machine learning to prevent overfitting and improve model generalization.



H

Figura 4: Training and Validation Error vs. Polynomial Degree M

13.1 What is Regularization?

Definizione 13.1 (Regularization)

Regularization refers to techniques that are used to calibrate machine learning models in order to minimize the adjusted loss function and prevent overfitting or underfitting.

In practice, regularization is a modification of the training error function by appending a term $\omega(f)$ that typically penalizes complex solutions.

13.2 The regularized objective function

The regularized objective function has the following form:

$$E_{reg}(f; \mathcal{D}_n) = E(f; \mathcal{D}_n) + \lambda_n \omega(f)$$

where:

- $E(f; \mathcal{D}_n)$ is the **training error function** (e.g., MSE)
- $\omega(f)$ is the **regularization term** that penalizes model complexity
- λ_n is the **tradeoff hyperparameter** that controls the strngth of regularization

The regularization term acts as a penalty that discourages the model from becoming too complex. By minimizing this combined objective, we find models that fit the data well while remaining relatively simple.

13.3 Effect of Regularization

The regularization term $\omega(f)$ typically measures some notion of model complexity. When we minimize the regularized objective:

- We balance fitting training data (low $E(f; \mathcal{D}_n)$) with keeping the model simple (low $\omega(f)$)
- The hyperparameter λ_n controls this tradeoff
- Higher λ_n means stronger regularization (simpler models)
- Lower λ_n means weaker regularization (more complex models allowed)

13.4 Regularization in Polynomial Fitting

For the polynomial fitting example, we can regularize by penalizing polynomials with large coefficients. The regularized error function becomes:

$$E_{reg}(f_w; \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^{n} [f_w(x_i) - y_i]^2 + \frac{\lambda}{n} \parallel w \parallel^2$$

where:

- The fisrt term is the training erorr (MSE)
- $\frac{\lambda}{n} \parallel w \parallel^2 = \frac{\lambda}{n} \sum_{j=0}^{M} w_j^2$ is the regularization term (L2 regularization)
- λ is the tradeoff hyperparameter

13.4.1 Effect of the Regularization Parameter λ

The choice of λ significantly affects the model:

- $\lambda \approx 10^{-18}$ (**very small**): Almost no regularization. The model can have large coefficients and may overfit. The polynomial fits the training data very closely, including noise.
- $\lambda = 1$ (**moderate**): Balanced regularization. The model is penalized for having large coefficients, leading to amoother curve that generalizes better. This often provides the best tradeoff between training and validation error.
- λ **very large**: Strong regularization. The penalty for large coefficients is so high that the model becomes too simple and may underfit. The curve becomes nearly flat.

13.4.2 Regularization Path

The plot of error vs. $ln(\lambda)$ shows:

- **Left side (small λ)**: Low training error, high validation error $\rightarrow$ Overfitting
- **Middle region:** Both errors are low $\rightarrow$ Good fit
- **Right side (large λ)**: Both errors increase $\rightarrow$ Underfitting

The optimal λ is found where the validation error is minimized.

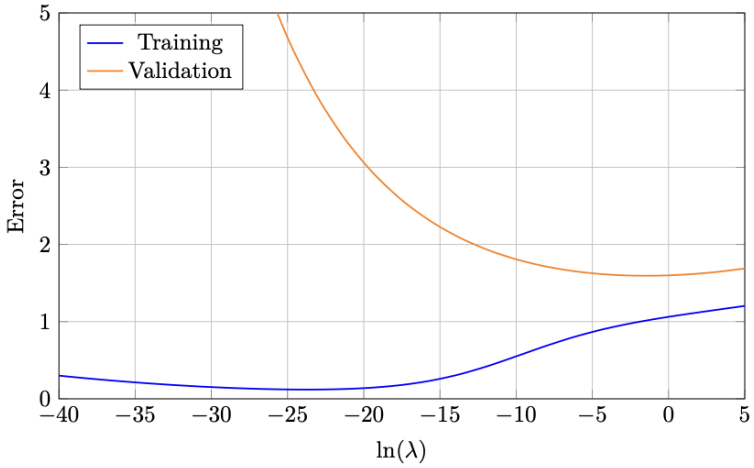


Figura 5: Training and Validation Error vs. $ln(\lambda)$ (Regualarization Strength)

13.5 Generalization and Data Size

An important theoretical result in machine learning relates training erorr to generalization error:

Teorema 13.2 (Generalization with Infinite Data)

As the number of training samples approaches infinity, the training error approzimates the generalization erorr:

$$E(f; \mathcal{D}_n) \to E(f; p_{data}) \text{ as } n \to \infty$$

This means:

- With a small dataset ($n = 15$), even a complex model ($M = 9$) may overfit because there isn't enough data to constrain the parameters
- With a large dataset ($n = 100$), the same complex model can generalize well beacouse the abundant data prevents overfitting
- More data allows us to use more complex models without overfitting

Visual example:

- $n = 15, M = 9$: The 9th-degree polynomial overfits, oscillating widly between the few training points
- $n = 100, M = 9$: With 100 training points, the same 9th-degree polynomial fits smoothly and generalizes well

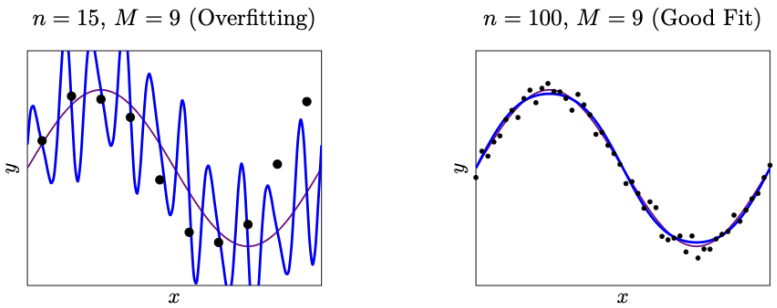


Figura 6: Effect of data size on generalization: With fex data points ($n = 15$), a complex model ($M = 9$) overfits. With many data points ($n = 100$), the same model generalizes well. Purple: true function, Blue: fitted polynomial, Black dots: trining data.

"Get mora data" is often the most effective solution to overfitting. With sufficinet data, even complex models can generalize well.

Parte II
Model Evaluation and Regression Methods

14 Model evaluation for Classification and Regression

14.1 Recap: Train, Validation, and Test Sets

The machine learning workflow involves three distinct datasets, each serving a specific purpose:

14.1.1 The three dataset

1. **Training set**: Used to train the machine learning algorithm
    - The model learns patterns and relationships from this data
    - Parameters are optimized to minimize training error
2. **Validation set**: Used during the learning phase
    - Also called the *mini test set*
    - Helps select the better performing algorithm among alternatives
    - Used to tune hyperparameters of the model

- Provides feedback to adjust the model before final testing

3. **Test set**: User for final model evaluation
    - Provides an unbiased assessment of model perfromance
    - The model has never seen this data during training or validation
    - Tells us how well the model will perform on new, real-world data

14.2 The importance of the validation set

The validation set plays crucial role in the machine learning pipeline and must kept **separate from the training set**.

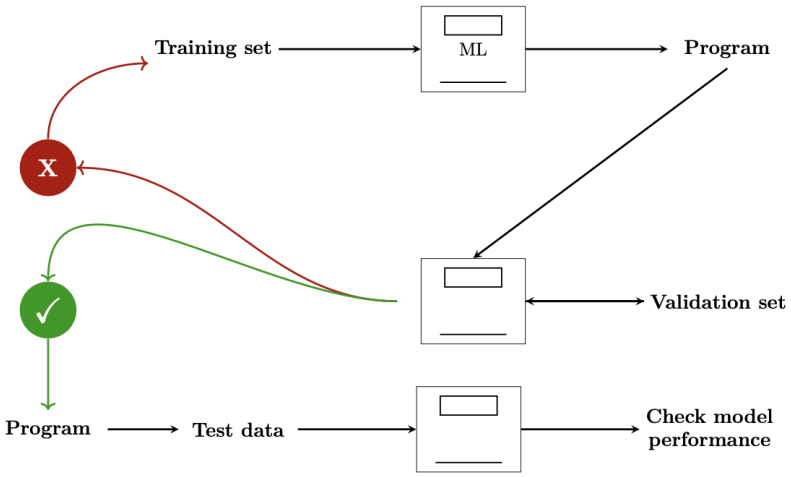


Figura 7: Machine learning workflow. Training set trains the model, validation set evaluates and provides feedback (thumbs down = retain, thumbs up = proceed to testing), test set checks final perfromance

14.2.1 Why separate the validation set?

The validation set is used for two main purposes:

1. **Algorithm Selection**: To pick the better performing algorithm
    - Compare different model architectures (e.g., decision trees vs. neural networks)
    - Select the model that performs best on unseen data
2. **Hyperparameter Tuning**: To decide the hyperparameters of an algorithm
    - Hyperparameters are configuration setting that are not learned from data
    - Examples: learning rate, regularization strength (λ), number of layers, tree depth

- The validation set helps find the optimal hyperparameter values

Note. **ATTENTION**: Splitting the datasets into training and validation sets can be done randomly to avoid bias. However, tehre are some **specific rules** to apply when performing this split to ensure reliable model evaluation.

14.3 Specific Rules for Dataset Splitting

When splitting data into training, validation, and testing sets, you can have conflicting priorities. The key is to balance these priorities appropriately.

14.3.1 Priority 1: Accurate Error Estimation

Goal: Estimate future error (i.e., validation/testing error) as accurately as possible.

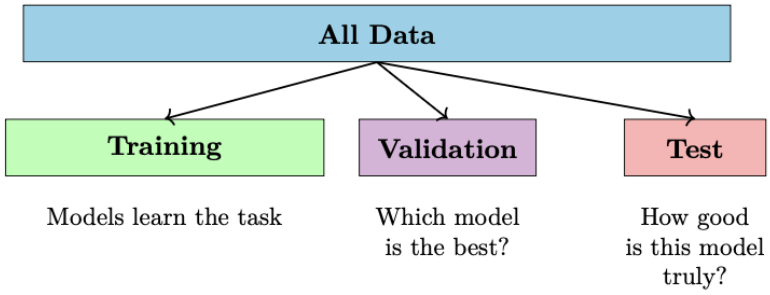


Figura 8: Dataset splitting: Training for learning, Validation for model selection, Test for final evaluation

How to achieve this: By making the **validation set as big as possible**.

(High confidence interval)

- A larger validation set provides more reliable error estimates
- Reduces variance in performance metrics
- Gives higher confidence that the measured perfromance reflects true generalization

14.3.2 Priority 2: Accurate Classifier Learning

Goal: Learn classifier as accurately as possible.

How to achieve this: By making the **training set as big as possible**

(Better estimates, maybe better generalization)

- More training data allows the model to learn more robust patterns

- Reduces overfitting by providing diverse examples
- Generally leads to better generalization on unseen data

14.3.3 The critical Rule

CRITICAL REQUIREMENT → Training and validation/testing instance **CANNOT OVERLAP !!!**

- If the same data appears in both training and validation/test sets, the evaluation is invalid
- The model would be tested on data i has already seen during training
- This lead to overlay optimistic performance estimates that don't reflect real-world performance
- Data leakage between sets is one of the most common mistakes in machine learning

Observation. The split between training and validation/test sets represents a fundamental trade-off:

- Larger training set → Better model learning, but less reliable error estimation
- Larger validation/test set → More reliable error estimation, but potentially worse model learning

Common splits include 70-15-15, 80-10-10, or 60-20-20 (train-validation-test), depending on the total dataset size and specific requirements.

15 Cross Validation

15.1 When do we need Cross Validation?

In cases where we don't have enough data to randomly split between training (e.g., 60% of the totale data), validation (e.g., 20% of the data), and testing (e.g., 20% of the totale data), we need to use **cross-validataion**.

Definizione 15.1 (Cross Validation)

Cross-Validation is a resampling technique that involves reusing a portion of the training set itself to validate performance by retraining not just one, but N models.

15.2 Key characteristics of Cross Validation

- **Multiple models**: Cross-Validation involves training N models instead of just one
- **Computional cost**: Performing cross-validation requires N training sessiones and can be more computationally demanding
- **Better use of limited data**: When data is scarce, cross-validation, allows us to use the available data more efficently for both training and validation

15.3 Fundamental Rules of Cross Validation

Cross validation must respect certain constraints to ensure valid model evaluation:

1. **No overlap**: Training and validation cannot overlap, but $n_{train} + n_{validation}$ = constant
2. **Each instance used once per fold**: At each fold (step), you use each instance only in one set, so there is no overlapping within signle fold
3. **Every sample participates**: Every sample is in both training and testing but not at the same time
    - This reduces the changs of getting a biased training set
    - Ensures all data contributes to both training and validation across different folds

15.4 K-Fold Cross Validation

K-fold cross validation is the most common form of cross validation, where the dataset is divided into k equal parts (folds).

15.4.1 How K-Fold Cross Validation works

In the case of k-fold cross-validation:

1. We split out dataset into k groups (folds)
2. We perform training k times on $(N/k)(k-1)$ data points
3. We measure performance on the last N/k data points (the validation fold)
4. Our cross-validation performance will be the **average of the perfromance of the k tests**

Note. **Important constraint**: Training set and validation set cannot overlap, but the sum of training and validation data is constant (the totale dataset size).

15.4.2 Visual Representation of K-Fold Cross Validation

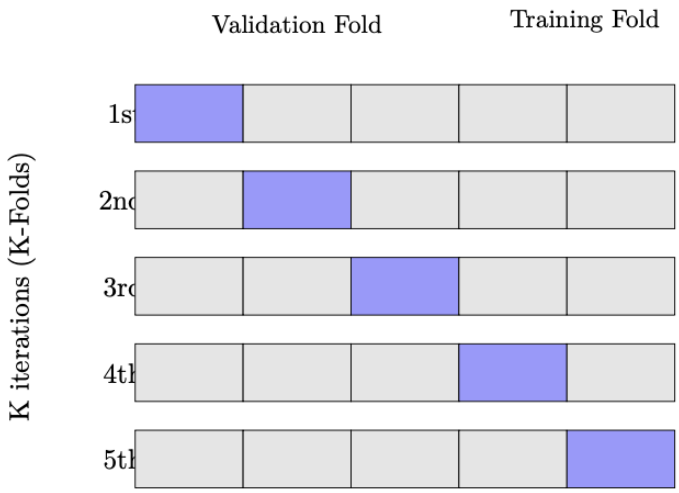


Figura 9: K-Fold Cross Validation: Each row represents on iteration where a different fold serves as validation (blue) while the rest serve as training (gray)

15.5 Example: 5-Fold Cross Validation

Let's examine a concrete example with 5 folds:

- **Step 1**: Randomly split the data into 5 folds
- **Step 2**: Test on each fold while training on 4 other folds (80% train, 20% test)
- **Step 3**: Average the results over 5 folds



Figura 10: 5-Fold Cross Validation Example: Each fold serves as validation once, and the final perfromance is the average of all 5 iterations

15.5.1 The process in Detail

For each of the 5 iterations:

1. **Iteration 1**: Use fold 1 as validation, folds 2-5 as training → Get Performance 1
2. **Iteration 2**: Use fold 2 as validation, folds 1,3-5 as training → Get Performance 2
3. **Iteration 3**: Use fold 3 as validation, folds 1-2,4-5 as training → Get Performance 3
4. **Iteration 4**: Use fold 4 as validation, folds 1-3,5 as training → Get Performance 4
5. **Iteration 5**: Use fold 5 as validation, folds 1-4 as training → Get Performance 5

Final Cross-Validation Performance:

$$\text{CV Performance } = \frac{1}{5}\sum_{i=1}^{5} \text{Performance}_i$$

15.6 Advantages of Cross Validation

- **Better use of limited data**: Every data point is used for both training and validation
- **More reliable performance estimates**: Averaging over multiple folds reduces vairance in the performance estimate
- **Reduces bias**: Every sample participates in validation, reducing the risk of biased validation set
- **Detects overfitting**: If performance varies significantly across folds, it may indicate overfitting or data quality issues

15.7 Disadvantages of Cross Validation

- **Computationally expensive**: Requires training k models instead of one
- **Time-consuming**: Training time is multiplied by the number of folds
- **Not suitable for very large datasets**: When a data is abundant, a simple train-validation-test split is more efficient

Observation. The choice of k (number of folds) involves a tradeoff:

- **Larger** k: More accurate performance estimate, but more computationally expensive
- **Smaller**: k: Faster computation, but less reliable performance estimate

- Common choices: $k = 5$ or $k = 10$.
- Extreme case: $k = N$ (Leave-One-Out Cross Validation) uses each single sample as validation once

16 Leave-One-Out Cross Validation

16.1 What is Leave-One-Out Cross Validation?

Leave-One-Out Cross Validation (LOOCV) is an extreme case of k-fold cross validation that is particularly useful when we have very few data points.

Definizione 16.1 (Leave-One-Out Cross Validation)

LOOCV is n-fold cross validation where n equals the total number of samples. In each iteration, we train on all $(n-1)$ samples and test on exactly 1 instance.

16.2 How LOOCV Works

The process involves:

1. Take the entire dataset of n samples
2. For each sample i (where $i = 1, 2, \ldots, n$):
   - Use sample i as the validation set (1 sample)
   - Use all other samples $(n-1)$ as the training set
   - Train the model and evaluate on the single validation sample
3. Repeat this process n times (once for each sample)
4. Average the performance across all n expriments

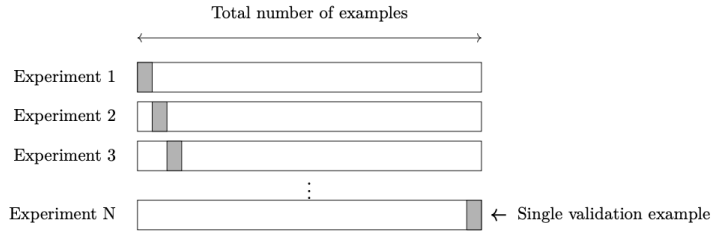


Figura 11: Leave-One-Out Cross Validation: Each experiment uses exactly one sample for validation (gray) and all others for training (white)

16.3 Pros and Cons of LOOCV

16.3.1 Advantages

- **Best possible classifier**: The model is learned from $n-1$ training examples, which is the maximum possible training data
- **Maximizes training data**: Each model uses almost all available data for training
- **Unbiased estimate**: Provides a nearly unbiased estimate of model performance
- **Deterministic**: No randomness in the splitting process (every sample is used exactly once as validation)

16.3.2 Disadvantages

- **High computational cost**: Must re-learn everything for n times, where n is the total number of samples
- **Time-consuming**: For large datasets, LOOCV becomes impratical
- **High variance**: Performance estimates can have high variance, especially with small datasets

Note. LOOCV is most appropriate when:

- The dataset is very small (e.g., $n < 50$)
- Computational resources are available
- You need the most accurate performance estimate possible

For larger datasets, standard k-fold cross validation (with $k = 5$ or $k = 10$) is typically preferred due to its better balance between computation cost and performance estimation accuracy.

17 Stratification in Cross Validation

17.1 The problem: Imbalanced Classes

In many real-world datasets, class labels are not evenly distributed. For example, in a medica diagnosis dataset, healthy patients might vastly outnumber sick patients. When performing cross validation on such imbalanced datasets, random splitting can lead to folds with very different calss distribution.

17.2 What is Stratification?

Definizione 17.1 (Stratification)

Stratification is a technique to keep class labels **balanced across training and validation sets** during cross validation. Instead of randomly diving the dataset, we ensure that each fold maintains approximately the same proportion

of samples for each class as the complete dataset.

17.3 How Stratified K-Fold Works

The stratification process works as follows:

1. **Divide by class**: Instead of taking the dataset and dividing it randomly into K parts, take the dataset and divide it into individual classes
2. **Split each class**: Then for each class, divide the instances into K parts
3. **Assemble folds**: Assemble the i-th part from alla classes to make the i-th fold
4. **Important**: It still random! The splitting within each class is random, but the proportions are maintained

17.4 Example: Stratified 5-Fold Cross Validation

Let's consider a concrete scenario with an imbalanced dataset containing 300 samples, divided into three classes (A, B, and C), where Class A has more samples than the other two.

17.4.1 Imbalanced Dataset

- **Class A**: 200 samples (66.7%).
- **Class B**: 50 samples (16.7%).
- **Class C**: 50 samples (16.7%).
- **Total**: 300 samples

17.4.2 Split the Dataset into Five Folds with Stratification

Each fold will have 60 samples total, maintaining the class proportions:

- **Fold 1**: 60 samples (Class A: 40, Class B: 10, Class C: 10)
- **Fold 2**: 60 samples (Class A: 40, Class B: 10, Class C: 10)
- **Fold 3**: 60 samples (Class A: 40, Class B: 10, Class C: 10)

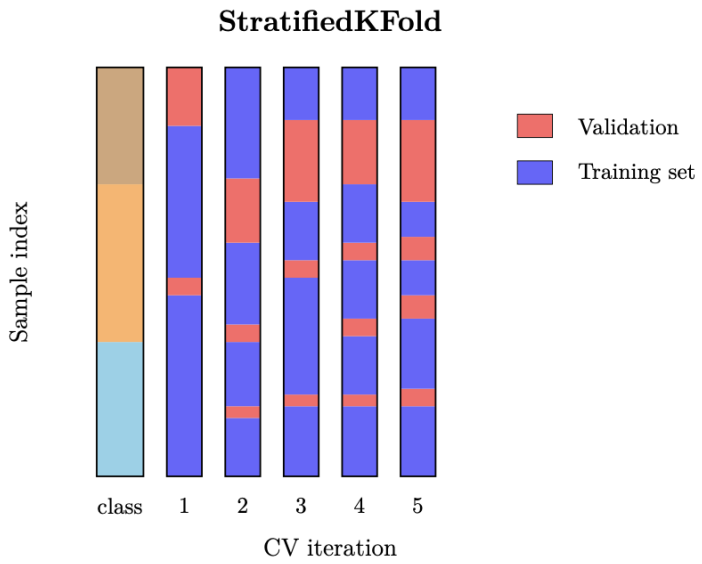


Figura 12: Stratified K-Fold: Each fold maintains the same class proportions across all folds

- **Fold 4**: 60 samples (Class A: 40, Class B: 10, Class C: 10)
- **Fold 5**: 60 samples (Class A: 40, Class B: 10, Class C: 10)

Notice that each fold maintains the same $66.7\% - 16.7\% - 16.7\%$ class distribution as the original dataset.

17.4.3 Train and Validate

The cross-validations proceeds as follows:

1. **Iteration 1**: Train on Folds 1, 2, 3, and 4, Validate on Fold 5
2. **Iteration 2**: Train on Folds 2, 3, 4, and 5, Validate on Fold 1
3. **Iteration 3**: Train on Folds 1, 3, 4, and 5, Validate on Fold 2
4. **Iteration 4**: Train on Folds 1, 2, 4, and 5, Validate on Fold 3
5. **Iteration 5**: Train on Folds 1, 2, 3, and 5, Validate on Fold 4

17.4.4 Average the performance Metrics

- Calculate performance metrics for each iteration
- Average these metrics to evaluate the model's performance robustly

17.5 Why Stratification Matters

- **Prevents biased folds**: Without stratification, some folds might have very few (or no) examples of minority classes
- **Consistent evaluation**: Each fold represents the overall dataset distribution, leading to more reliable performance estimates
- **Better for imbalanced datasets**: Particularly important when dealing with class imbalance
- **Reduces variance**: Performance estimates have lower variance across folds

Note. **When to use stratification**:

- Always use stratification for classification problems with imabalanced classes
- It's generally a good practice even for balanced datasets
- Most machine learning librearies (e.g., scikit-learn) provide stratified cross-validation functions
- For regression problems, stratification is not applicable (since there are no discrete classes)

Observation. Stratification ensures that:

- Each fold is a good representative of the whole dataset
- Minority classes are adequately represented in both training and validation sets
- Performance metrics are more stable and reliable across folds
- The model ses a balanced distribution of classes during each training iteration

18 Evaluation Measures

Once we have trainend out models and performed cross-validation, we need to evaluate their performance. Different tasks require different evaluation metrics.

18.1 Why do we need Evaluation Measures?

Evaluation measures serve two primary purpose:

1. **To decide if our model is performing well**: Assess wether the model meets the requirements for the task
2. **To decide out of many models, which one performs better than the other**: Compare different models and select the best one for deployement

18.2 Evaluation Measures by Task Type

Different nachine learning tasks require different evaluation approaches:

18.2.1 Classification

Question: How often does our model classify something right/wrong?

Classification metrics measure the correctness of predictions, focusing on whether the predicted class matches the actual class.

18.2.2 Regression

Question: How close our model to what we are trying to predict?

Regression metrics measure the distance or error between predicted continuos values and avtual values.

18.2.3 Clustering

Question: How well does our model describe our data?

Clustering evaluation is generally very hard because there's often no ground truth to compare against. Metrics must assess the quality of discovered patterns.

19 Evaluation Measures of Classification

19.1 Types of Classification Problems

A classification model takes a set of features as input and produces one or more classes as output.

Definizione 19.1 (Classification Type)

Classification can be categorized into different types based on the number of classes:

- **Binary classification**: In the case of two classes (0/1) or one vs rest (e.g., apples vs other fruits)
- **Multi-class Classification**: In the case having N classes to choose from (e.g., cat, dog, wolf, etc.)
- **Multi-label Classification**: A problem where an example can belong to multiple calsses simultaneously

19.2 The confusion matrix

The confusion matrix is a fundamental tool for evaluating binary classification models. It provides a complete picture of how the model perfrom across different types of predictions

19.2.1 Confusion Matrix for Binary classification

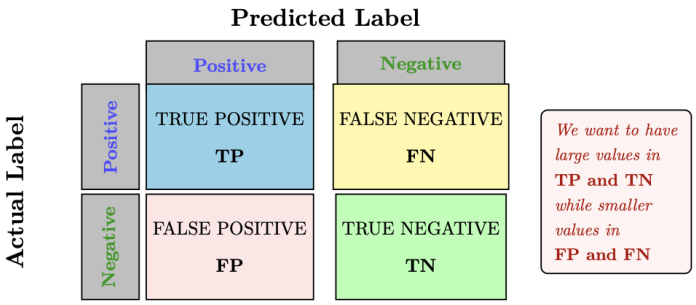


Figura 13: Confusion Matrix for Binary Classification

19.2.2 Understanding the Confusion Matrix

The confusion matrix has four components:

- **True Positive (TP)**: The model correctly predicted the positive class
  - Actual label: Positive
  - Predicted label: Positive
  - Good prediction!
- **True Negative (TN)**: The model correctly predicted the negative class
  - Actual label: Negative
  - Predicted label: Negative
  - Good Prediction!
- **False Positive (FP)**: The model incorrectly predicted positive (Type I error)
  - Actual label: Negative
  - Prediction label: Positive
  - Bad prediction! (False alarm)
- **False Negative (FN)**: The model incorrectly predicted negative (Type II error)
  - Actual label: Positive
  - Prediction label: Negative
  - Bad prediction! (Missed detection)

Note. **Goal**: We want to maximize TP and TN (correct predictions) while miniimzing FP and FN (incorrect predictions).

19.3 Classification Error and Accuracy

From the confusion matrix, we can derive several important metrics.

19.3.1 Classification error

Definizione 19.2 (Classification Error)

The classification error (also called misclassification rate) measures the proportion of incorrect predictions:

$$\text{Classification Error } = \frac{FP + FN}{TP + TN + FP + FN}$$

This represent the fraction of all predictions that were wrong

19.3.2 Accuracy

Definizione 19.3 (Accuracy)

Accuracy measures the proportion of correct predictions:

$$\text{Accuracy } = 1 - \text{Error} = \frac{TP + TN}{TP + TN + FP + FN}$$

This represent the fraction of all predictions that were correct

19.4 The problem with Accuracy: Imbalanced Classes

While accuracy is a basic measure of "goodness" of a classifier, it can be very misleading when dealing with imbalanced classes.

ATTENTION! $\rightarrow$ Accuracy is **very misleading** if we have **imbalanced classes**

19.4.1 Example: The imablanced Dataset Problem

Consider a dataset with severe class imbalance:

- 90 samples for class "NO"
- 10 samples for class "YES"

Native strategy: A trivial classifier that predicts "NO" for every sample would achieve:

$$\text{Accuracy } = \frac{90}{100} = 90\%$$

The problem: Despite having 90% accuracy, this model is completely useless! It cannot classify any data belonging to the "YES" class. The model has learned nothing meaningful about the data.

Observation. The example illusatrate why accuracy alone is insufficient for evaluating classifiers, especially with imablanced dataset. We need additional metrics that provide a more nuanced view of model performance across all classes.

Note. When dealing with imbalanced dataset, always consider:

- Using stratified cross-validation
- Examining the confusion matrix in detail
- Using additional metrics beyond accuracy (precision, recall, F1-score)
- Considering class-specific performance
- Using techniques like oversampling, undersampling, or class weights

19.5 Misses and False Alarms

Beyond accuracy, we need more specific metrics to understand model performance. These metrics focus on different aspects of classification errors.

19.5.1 False Alarm Rate (False Positive Rate)

Definizione 19.4 (False Alarm Rate = False Positive Rate)

$$\frac{FP}{FP+TN}$$

This represents the *percentage of negative samples we misclassified as positive*

19.5.2 Miss Rate (False Negative Rate)

Definizione 19.5 (Miss Rate = True Positive Rate)

$$\frac{TP}{TP+FN}$$

This represent the *percentage of positive samples we misclassified as negative*

19.5.3 Recall (True Positive Rate)

Definizione 19.6 (Recall = True Positive Rate)

$$\frac{TP}{TP+FN}$$

This represent the *percentage of positive samples we classified correctly*

It also know as **Sensitivity** and can be expressed as: Recall = 1 - Miss Rate

Observation. Recall answers the question: "Of all the actual positive cases, how many did we identity?"

19.5.4 Precision

Definizione 19.7 (Precision)

$$\frac{TP}{TP+FP}$$

This represent the *percentage positive out of what we predicted was positive.*

Observation. Precision answer the question: "Of all the cases we predicted as positive, how many were actually positive?"

19.6 Cost of the Task

In pratice, we typically do not use evaluation mertics like accuracy, recall, or precision alone. Instead, we declare a combination of them together based on the specific requirements of the task

19.6.1 The need for a single evaluation measure

- In oreder to optimize a learner automatically (i.e., during training), we need a **single evaluation measure**
- How do we decide that single metric?
  - **Domain specific!** - It depends on the task

19.6.2 weighting Errors by Cost

Depending on the **cost of the task**, we can decide whether we take more about having:

- *Less false positives*, or
- *Less false negatives*

This is achieved through wighting them:

$$\text{Cost} = C_{FP} \times \text{FP} \ + C_{FN} \times \text{FN}$$

where C_{FP} and C_{FN} are the costs associated with false positive and false negatives, respectively.

Esempio 19.8 (Medical Diagnosis (Cancer detection))

- False Negative (missing a cancer diagnosis): Very high cost - patient doesn't receive treatment
- False Positive (false alarm): Lower cost - patient undergoes additional tests
- Therefore: $C_{FN} \gg C_{FP}$, we prioritize high recall

Esempio 19.9 (Spam Email Filter)

- False Positive (marking important email as spam): High cost - user misses important messages
- False Negative (spam gets through): Lower cost - minore annoyance
- Therefore: $C_{FP} > C_{FN}$ we prioritize high precision

19.7 F-Measure (F1 Score)

The F-Measure combines precision and recall into a single metric, providing a balance between the two.

Definizione 19.10 (F1)

The **F1 Score** is the harmonic mean of precision and recall:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Harmonic mean of precision and recall, one of the most frequently used metrics in machine learning

19.7.1 Properties of F1 Score

- $(+)$ If you do some mathematics, you will see that the F1 measure is sort of an accuracy without TN
- $(+)$ Used frequently in information retrieval system
- $(+)$ Provides a singke number that balances precision and recall
- **Range**: $F_1 \in [0, 1]$, where 1 is perfect and 0 is the worst

Note. The harmonic mean (used in F1) is always less than or equal to the arithmetic mean. This means F1 score give more weight to the lower value between precision and recall, ensuring both metrics need to be reasonably high for a good F1 score.

19.7.2 Generalized F-Measure

The F1 score can be generalized to F_β score, which allows adjusting the relative importance of precision and recall:

$$F_\beta = (1 + \beta^2) \times \frac{\text{Precision} \times \text{Recall}}{\beta^2 \times \text{Precision} + \text{Recall}}$$

where:

- $\beta < 1$: Emphasizes precision
- $\beta > 1$: Emphasizes recall
- $\beta = 1$: Equal weight (standard F1 score)

19.8 Multiclass Classification - Evaluation

For multiclass classification problems (more than two classes), we need to extend our evaluation metrics.

19.8.1 Multiclass confusion matrix

- n_{ij} is the number of examples with actual label (true label) y_i and predicted label y_j.
- The **main diagonal** contains true positive for each class
- The **sum of off-diagonal elements along a column** is the number of false positives for the column label
- The **sum of off-diagonal elements along a row** is the number of false negatives for the row label

19.8.2 Multiclass Metrics

For each class i, we can compute:

$$FP_i = \sum_{j \neq i} n_{ij} \quad FN_i = \sum_{j \neq i} n_{ij}$$

$$\text{Precision }_i = \frac{n_{ii}}{n_{ii} + FP_i} \quad \text{Recall}_i = \frac{n_{ii}}{n_{ii} + FN_i}$$

$$\text{Multiclass Accuracy (MAcc)} = \frac{\sum_i n_{ii}}{\sum_i \sum_j n_{ij}}$$

19.8.3 Macro vs. Micro averaging

When reporting overall metrics for multiclass problem, we have two main approaches:

- **Macro-averaging**: Compute metric for each class independently, then take the average

  $$\text{Macro-Precision} = \frac{1}{C} \sum_{i=1}^{C} \text{Precision}_i$$

  This gives equal weight to each class regardless of size

- **Micro-averaging**: Aggregate the contributions of all classes to compute the average metric

  $$\text{Micro-Precision} = \frac{\sum_i TP_i}{\sum_i (TP_i + FP_i)}$$

  This gives equal weight to each sample

19.9 ROC Curve - Receiver Operating Characteristic

The ROC curve is a powerful tool for evaluating and comparing binary classifiers, especially when the decision threshold can be varied.

19.9.1 What is the ROC curve?

The **ROC curve** is a graphical plot that illustrates the diagnostic ability of a binary classifier as its discrimination threshold is varied.

- **X-axis**: False Positive Rate (FPR) = $\frac{FP}{FP+TN}$.
- **Y-axis**: True Positive Rate (TPR) = Recall = $\frac{TP}{TP+FN}$.

19.9.2 Uses of ROC Curves

The ROC curve allows us to:

- **Compare different classifiers**: Different models can be plotted on the same ROC space
- **Estimate the false positive rate / negative rate depending on the threshold** that we set to classify
- Visualize the trade-off between sensitivity (recall) and specificity

19.9.3 Understanding the ROC space

- **Perfect classifier**: A point at $(0, 1) - 100\%$TPR $, 0\%$ FPR.
- **Random classifier**: Lies on the diagonal line from $(0, 0)$ to $(1, 1)$.
- **Better classifier**: Curves that bow toward the upper left corner
- **Worse classifier**: Curves below the diagonal (worse than random guessing)

19.9.4 Area under the ROC Curve (AUC)

Definizione 19.11 (Area under ROC Curve)

The **Area under the ROC Curve (AUC)** is a single scalar value that summarize the performance of a classifier across all possible thesholds

$$AUC \in [0, 1]$$

Goal: Maximize the Area under the ROC Curve (AUC)

19.9.5 Interpreting AUC Values

- **AUC = 1.0**: Perfect classifier
- **AUC = 0.9 - 1.0**: Excellent
- **AUC = 0.8 - 0.9**: Good
- **AUC = 0.7 - 0.8**: Fair
- **AUC = 0.6 - 0.7**: Poor
- **AUC = 0.5**: No better than random guessing

- **AUC < 0.5**: Worse than random (predictions are inverted)

Observation: The AUC can be interpreted as the probability that the classifier will rank a randomly chosen positive instance higher that a randomly chosen negative instance

19.9.6 Advantages of ROC and AUC

- **Threshold-independent**: Evaluates the model across all possible threshold
- **Class imbalance robust**: Unlike accuracy, AUC is less sensitive to class imbalance
- **Single metric**: Provides a single value for model comparison
- **Visual interpretation**: The ROC curve provides intuitive visualization of classifier performance

20 Regression Evaluation Metrics

While classification tasks allow us to count correct and incorrect predictions, regression tasks require different evaluation approaches since we're predicting continuos values.

20.1 The Fundamental Difference

- **Classification**: We can count how often we are correct or wrong in out predictions
- **Regression**: We cannot do that counting!
  - Predicting y_i (not discrete, continuos value) from inputs x_i.
  - The question is not "*how many times your method is wrong*" but "*how much your method is wrong with respect to the groundtruth-labels*"

Key Difference:

$$\text{Classification} : \text{Count errors} \leftrightarrow \text{Regression} : \text{Measure error magnitude}$$

20.2 Mean Squared Error (MSE)

The mean squared error is one of the most commonly used metrics for regression task.

Definizione 20.1 (Mean Squared Error)

$$MSE(f) = \frac{1}{N}\sum_{i=1}^{N}(f(x_i) - y_i)^2$$

where:

- f is the model that takes a feature vector x as input and generates a

prediction $f(x_i)$.

- i, ranging from 1 to N, denotes the index of sample the dataset
- y_i is the ground truth (actual value)

20.2.1 Properties of MSE

- The sum of predictions minus real values over N indicates the average
- **Squaring** removes the negative sign and gives more weight to larger differences
- **The lower the MSE, the better**
- MSE is **influenced by outliers**, i.e., data values that are abnormally distant from the true regression line
- MSE is **differentiable**, which makes it suitable for gradient-based optimization algorithms

Observation: The squaring operation in MSE has two important effects:

1. It ensure all errors are positive (no cancellation between positive and negative errors)
2. It penalizes large errors more heavily that small errors (quadratic penalty)

20.3 Mean absolute error (MAE)

The Mean Absolute Error provides an alternative way to measure regression performance

Definizione 20.2 (Mean Absolute Error)

$$MAE(f) = \frac{1}{N}\sum_{i=1}^{N} \mid f(x_i) - y_i \mid$$

20.3.1 Properties of MAE

- MAE is similar to MSE in that it returns absolute values of the residual $f(x_i) - y_i$ without squaring
- It does not consider the direction of the error, which means we will not know whether negative or positive errors weigh more on the overall average
- **MAE is more robust to outliers** precisely due to the absence of squaring of distant forecast error values
- However, MAE is **not differentiable** at zero. It means that we cannot take the derivative of it, and taking derivatives is important if you want to build an algorithm that minimizes the function. Because of this, it used a lot less than MSE

MSE vs MAE Comparison

- **MSE**: Differentiable, penalizes large errors more, sensitive to outliers
- **MAE**: Not differentiable at zero, treats all errors equally, robust to outliers

20.4 Choosing Between MSE and MAE

The choice between MSE and MAE depends on the specific requirements of your regression task:

Metric	Use When	Avoid When
MSE	• Large errors are particularly undesirable • You need gradient-based optimization • Outliers should be heavily penalized	• Dataset contains many outliers • All errors should be treated equally
MAE	• Outliers are present in the data • All errors should be weighted equally • You want a more interpretable metric	• You need differentiability for optimization • Large errors should be penalized more

20.5 Other Regression Metrics

Beyond MSE and MAE, there are other useful metrics for evaluating regression models:

20.5.1 Root Mean Squared Error (RMSE)

$$RMSE(F) = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(f(x_i) - y_i)^2}$$

RMSE is simply the square root of MSE. It has the advange of being in the same units as the target variable, making it more interpretable.

20.5.2 R-Squared (R^2) Score

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - f(x_i))^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the observed values. R^2 represents the proportion of variance in the dependent variable that is predictable from the independent variables. It ranges from 0 to 1, with 1 indicating perfect predition.

Note. In pratice, it's often beneficial to report multiple metrics to get comprehensive view of model performance. For example, reporting both MSE (or RMSE) and

MAE can provide insights into both the typical error magnitude and the presence of outliers.

Parte III
Bayes Classifier and Nonparametric Classifiers

21 Bayes Classifier

21.1 Introduction

The bayes classifier represents a funamental **statistical approach** for classification problems. It provides an optimal decision-making framework when all relevant probabilities are known.

Teorema 21.1 (Core Hypothesis)

The decision problem is cast in **probabilistic terms**, where all relevant **probabilities are known**

21.1.1 Goal of Bayes Classifier

The primary objectives are:

- **Discriminate** between different decision using the probabilities and thei associated costs
- **Estimate** the **joint probability** $p(\mathbf{x}, y)$ from the training data set

Key Concept $\rightarrow$ The bayes classifier aims to estimate the join probability distribution $p(\mathbf{x}, y)$ to make optimal classification decisions.

21.2 An Easy Example

Let's start with a simple example to understande the basic principles.

Esempio 21.2 (Two-Class Classification)

Let y be what we want to classify, and it is to be probabilistically described.

There are **two classes** y_1 and y_2 which correspond to two possible states:

a) $\mathrm{P}(y = y_1) = 0.7$

b) $\mathrm{P}(y = y_2) = 0.3$

These are called **a-priori** or **prior probabilities**

Scenario: There are no measurements or obersations available to help make a decision about which class y belogns to.

21.2.1 Decsion Rule

Given only the prior probabilities, the optimal decision rule is straight forward: **Decision Rule** $\rightarrow$ Decide y_1 if $P(y_1) > P(y_2)$ otherwise decide y_2.

Observation. Given the probabilities (70% for y_1 and 30% for y_2), the decision rule would lead to classifying y as y_1 since 0.7 is greater than 0.3.

This makes intuitive sense: without any additional information, we should choose the more probable class.

21.3 Another example: Adding Measurements

Now let's consider a more realistic scenario where we have measurements to help us decide.

Esempio 21.3 (Classification with Measurements)

Having the previous hypothesis and additionally **a single measurement x**, which is a random variable that depends on the class y_j, we can get:

$p(\mathbf{x} \mid y_j)_{j=1,2} =$ **Likelihood** or **class-conditional probability density function**

i.e., *the probability of having the measurement* $\mathbf{x}$ *knowing that y is* y_j

Observation. When we fix the measurement $\mathbf{x}$, the higher $p(\mathbf{x} \mid y_j)$ is, the more likely y_j is the correct state.

This is the key insight: measurements provide evidence that can shift our belief from the prior probabilities toward the class that better explains the observed data.

21.3.1 Visual interpretation

The class-conditional probability density functions can be visualized as distributions over the measurement space. For two classes:

- Each class has its own probability in distribution $p(\mathbf{x} \mid y_j)$.
- These distributions may overlap in the measurement space
- Where they overlap, we need a decision rule to determine which class is more likely
- The shape and position of these distributions reflect how the measurement $\mathbf{X}$ is distributed for each class

21.4 Bayes Formula

The Bayes formula provides the mathematical framework for combining prior knowledge with observed evidence.

Definizione 21.4 (Bayes Formula)

Assuming known $P(y_j)$ and $p(\mathbf{x} \mid y_j)$, the decision of ω becomes, for Bayes:

$$p(y_j, \mathbf{x}) = \mathcal{P}(y_j \mid \mathbf{x}) = p(\mathbf{x} \mid y_j)\mathcal{P}(y_j)$$

that is:

$$\mathcal{P}(y_j \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid y_j)\mathcal{P}(y_j)}{p(\mathbf{x})} \propto p(\mathbf{x} \mid y_j)\mathcal{P}(y_j)$$

where:

- $\mathcal{P}(y_j)$ = **Prior**
- $p(\mathbf{x} \mid y_j)$ = **Likelihood**
- $\mathcal{P}(y_j \mid \mathbf{x})$ = **Posterior**
- $p(\mathbf{x}) = \sigma_{j=1}^{J} p(\mathbf{x} \mid y_j)\mathcal{P}(y_j)$ = **Evidence**

21.4.1 Understanding the components

Definizione 21.5 (Bayes formula Components)

- **Prior**: $\mathcal{P}(y_j)$: Our initial belief about the probability of class y_j before seeing any data
- **Likelihood**: $p(\mathbf{x} \mid y_j)$: The probability of observing measurement $\mathbf{x}$ given that the true class is y_j.
- **Posterior**: $\mathcal{P}(y_j \mid \mathbf{x})$: Our updated belief about the probability of class y_j after observing measurement $\mathbf{x}$.
- **Evidence**: $p(\mathbf{x})$: The total probability of observing measurement $\curvearrowleft$ across all classes (acts as a normalization constant)

Observation. The evidence $p(\mathbf{x})$ is the same for all classes, which is why we can use the proportionality:

$$P(y_j \mid \mathbf{x}) \propto p(\mathbf{x} \mid y_j)P(y_j)$$

This means for classification, we only need to compare $p(\mathbf{x} \mid y_j)P(y_j)$ across different classes without computing the evidence explicitly.

21.4.2 Bayes Decision Rule

Optimal Bayes Decision Rule

Choose class y_j that maximizes the posterior probability:

$$\hat{y} = \arg\max_j P(y_j \mid \mathbf{x}) = \arg\max_j p(\mathbf{x} \mid y_j\, P(y_j))$$

Note. The bayes classifier is optimal in the sense that it minimizes the probability of misclassification when all probabilities are known exactly. However, in practice,

we must estimate these probabilities from data, which introduces approximation errors.

21.5 Bayes Decision Rule: Detailed Analysis

Bayes Formula Relationship

$$\mathcal{P}(y_j \mid \mathbf{x}) = \frac{p(\mathbf{x} \min y_j)P(y_j)}{p(\mathbf{x})} \Leftrightarrow \text{ posterior } = \frac{\text{likelihood } \times \text{ prior}}{\text{evidence}}$$

21.5.1 Understanding the Posterior

- The posterior or **a-posteriori probability** is the probability that y is y_j given the observation $\mathbf{x}$.
- The most important factor is the product likelihood $\times$ prior
- The evidence $p(\mathbf{x})$ is simply a scale factor, which ensures that:

$$\sum_j P(y_j \mid \mathbf{x}) = 1$$

Observation. Since $p(\mathbf{x})$ is the same for all classes, it acts as a normalization constant. For classification purposes, we can ignore it and simply compare the numerators $p(\mathbf{x} \mid y_j)P(y_j)$ across different classes.

21.5.2 The Decision Rule

From the formula of Bayes derives the **Bayes's decision rule**:

Bayes Decision Rule

Decide y_1 if $\mathcal{P}(y_1 \mid \mathbf{x}) > \mathcal{P}(y_2 \mid \mathbf{x})$ and y_2 otherwise

Equivalentely, since we can ignore the evidence:

Decide y_1 if $p(\mathbf{x} \mid y_1)P(y_1) > p(\mathbf{x} \mid y_2)P(y_2)$, and y_2 otherwise.

22 Naïve Bayes Classifier

The Naïve Bayes Classifier is a pratical implementation of Bayes' theorem that makes a simplifying assumption about feature independence.

22.1 Core Principles

Definizione 22.1 (Naïve Bayes Classifier)

A probabilistic classification algorith based on **Bayes' theorem** that:

- Assumes **independence** among features
- Calculates the **posterior probability** using:

$$\mathcal{P}(y_j \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid y_j)\mathcal{P}(y_j)}{p(\mathbf{x})}$$

22.2 The Naïve Independence Assumption

The key assumption is that features $x_1, x_2, \ldots, x_n$ are **independent** given the class label ω:

$$\mathcal{P}(\mathbf{x} \mid y) = \mathcal{P}(x_1 \mid y)P(x_2 \mid y)\ldots P(x_n \mid y) = \prod_{i=1}^{n} \mathcal{P}(x_i \mid y)$$

Observation. This is called "naïve" because in reality, features are often correlated. However, despite this simplification, Naïve Bayes often performs surprisingly well in practice, especially for text classification tasks.

22.2.1 Training the Classifier

Key Point: Probability of each class is determined from training data.

The training process involves:

1. Estimating the prior probabilities $P(y_j)$ from the class frequencies in the training data
2. Estimating the conditional probabilities $P(x_i \mid y_i)$ for each feature given each class

22.3 Example: Email Spam Classification

Let's work through a complete example to understand how Naïve Bayes works in practice.

Esempio 22.2 (Spam detection)

We want to classify an email as either:

- **Class 1**: Spam
- **Class 2**: Not Spam (Ham)

We'll use a few features (words) that appear in the email.

22.3.1 Training Data

Consider the following training dataset:

Email	Contains "Free"?	Contains "Win"?	Class
E1	Yes	Yes	Spam
E2	Yes	No	Spam
E3	No	Yes	Spam
E4	No	No	Ham
E5	No	No	Ham

22.3.2 Step 1: Compute Class Priors

From the training data:

$$P(\text{Spam}) = \frac{3}{5}$$

$$P(\text{Ham}) = \frac{2}{5}$$

Observation. The prior probabilities reflect the class distribution in out training set: 3 out of 5 email are spam, and 2 out of 5 are ham.

22.3.3 Step 2: Estimate Conditional Probabilities

For each feature (word) and class, we estimate how often the word appears when the email belongs to that class:

For "Free":

$$P(\text{Free} = \text{Yes} \mid \text{Spam}) = \frac{2}{3}$$

$$P(\text{Free} = \text{Yes} \mid \text{Ham}) = \frac{0}{2} = 0$$

For "Win":

$$P(\text{Win} = \text{Yes} \mid \text{Spam}) = \frac{2}{3}$$

$$P(\text{Win} = \text{Yes} \mid \text{Ham}) = \frac{0}{2} = 0$$

Note. Note that we also need the probabilities for the words *not* appearing:

$$P(\text{Free} = \text{No} \mid \text{Spam}) = 1 - \frac{2}{3} = \frac{1}{3}$$

$$P(\text{Win} = \text{No} \mid \text{Spam}) = 1 - \frac{2}{3} = \frac{1}{3}$$

$$P(\text{Free} = \text{No} \mid \text{Ham}) = 1 - 0 = 1$$

$$P(\text{Win} = \text{No} \mid \text{Ham}) = 1 - 0 = 0$$

22.3.4 Step 3: Classify a New Email

Validation/Testing: Consider a new email that contains "Free" but does **not** contain "Win".

We compute (ignoring the denominator (evidence) since it's the same for both classes):

$$P(\text{Spam} \mid \text{Free, no Win}) \propto P(\text{Spam})P(\text{Free} \mid \text{Spam})P(\text{no Win} \mid \text{Spam})$$

$$= \frac{3}{5} \times \frac{2}{3} \times (1 - \frac{2}{3})$$

$$= \frac{3}{5} \times \frac{2}{3} \times \frac{1}{3} = \frac{2}{15}$$

$$P(\text{Ham} \mid \text{Free, no Win}) \propto P(\text{Ham})P(\text{Free} \mid \text{Ham})P(\text{no Win} \mid \text{Ham})$$

$$= \frac{2}{5} \times 0 \times 1 = 0$$

Prediction: The model predicts **Spam**, because its probability is greater.

$$P(\text{Spam} \mid \text{Free, no Win}) = \tfrac{2}{15} > P(\text{Ham} \mid \text{Free, no Win}) = 0$$

22.4 The zero probability Problem

Observation. Notice that $P(\text{Free} \mid \text{Ham}) = 0$ caused the entire probability for Ham to become zero. This is a common problem in Naïve Bayes called the **zero probability problem**.

22.4.1 Solution: Laplace Smoothing

To avoid zero probabilities, we use **Laplace smoothing** (also called additive smoothing):

$$P(x_i \min y_i) = \frac{\text{count}(x_i, y_i + \alpha)}{\text{count}(y_i) + \alpha \cdot n}$$

where:

- α is the smoothing parameter (typically $\alpha = 1$)
- n is the number of possibile values for feature x_i

Note. Laplace smoothing ensures that no probability is exectly zero, which prevents the entire product from becoming zero when a feature value hasn't been observed in the training data for a particular class.

22.5 Advantages and disadvantages

Advantages	Disadvantages
Simple and fast to train and predict	Independence assumption is often violated
Works well with high-dimensional data	Can be outperformed by more sophisticated models
Requires relatively small training data	Zero probability problem require smoothing
Performs well for text classification	Cannot learn feature interactions
Provides probabilistic predictions	Sensitive to irrelevant features

22.6 Classification Rule Summary

Naïve Bayes Classification Rule

Classify an observation $\mathbf{x}$ to class y_j if:

$$P(y_j \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid y_j)P(y_j)}{p(\mathbf{x})} \text{ is maximized}$$

22.6.1 Key Points

Advantages:

- Fast to train and predict, suitable for large datasets
- Works well with high-dimensional data

Limitations:

- Independence assumption may not hold in real-world scenarios, leading to suboptimal performance
- Zero probability problem: requires techniques (like Laplace Smoothing) for unseen feature-class combinations

22.7 Problems with Bayesian Classifier

To create an optimal classifier that uses the Bayesian decision rule, you need to know:

1. **Prior probabilities** $P(y_j)$.
2. **Class-conditional densities (likelihood)** $p(\mathbf{x} \mid y_j)$

Observation. The performance of a classifier **strongly** depends on the **godness** of these components.

The fundamental Problem

BUT PRATICALLY ALL THIS INFORMATION IS NEVER AVAILABLE!!

22.7.1 What we actually have

More often we only have:

- A vague knowledge of the problem, from which to extract vague **a-priori probabilities**
- Some particularly representative patterns, training data, used to train the classifier (**often too few or sparse!**)

Observation.

- Estimating a-priori probabilities is usually not particularly difficult
- Estimating conditional densities (likelihood) is **more complex** $\rightarrow$ needs approximations

22.8 Challenges in Estimating Conditional Densities

Estimating conditional densities (likelihood) is more complex and needs approximations due to several challenges:

1. **High-dimensional problem**: Data can have many features $\rightarrow$ exponential combinations
   - With d features, each with k possible values, there are k^d possible feature combinations
   - This leads to the **curse of dimensionality**
2. **Zero probabilities**: Some feature combinations may never appear in training data $\rightarrow$ zero probabilities
   - This can cause the entire posterior probability to become zero
   - Requires smoothing techniques
3. **Continuous features**: Features may be continuos, requiring parametric assumptions (e.g., Gaussian)
   - We need to assume a probability distribution family
   - Then estimate the parameters of that distribution

Key Challenge

The complexity of estimating $p(\mathbf{x} \mid y_j)$ grows exponentially with the number of features making it impratical to estimate directly from data without making simplifying assumptions.

22.9 Solution: Parametric Models

One solution (out of many) is to approximate the conditional densities by **fitting a parametric model**.

Definizione 22.3 (Parametric Approach)

We approximate the likelihood by fitting a **parametric model** (e.g., Gaussian) to the training data and estimate its parameters using, for example, **Maximum Likelihood Estimation (MLE)**.

- It is a standard way to estimate parameters associated to the likelihood from training data
- For a Gaussian model, the parameters are:

$$\theta_j = (\mu_j, \Sigma_j)$$

  where μ_j is the mean vector and Σ_j is the covariance matrix for class y_j.
- The likelihood is then approximated as:

$$P(\mathbf{x} \mid y_j) \approx \mathcal{N}(\mu_j, \Sigma_j)$$

22.9.1 Maximum Likelihood Estimation (MLE)

Definizione 22.4 (Maximum Likelihood Estimation)

MLE choose the parameter values that make the observed data **most probable**.

For a dataset $\mathcal{D} = \{(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \ldots, (\mathbf{x}_N, \mathbf{y}_N)\}$, the likelihood function is:

$$L(\theta_j) = \prod_{i:y_i=y_j} p(\mathbf{x}_i \mid y_j, \theta_j)$$

MLE finds:

$$\hat{\boldsymbol{\theta}}_j = \arg\max_{\boldsymbol{\theta}_j} L(\boldsymbol{\theta}_j)$$

Observation. In pratice, we often maximize the **log-likelihood** instead:

$$logL(\theta_j) = \sum_{i:y_i=y_j} logp(\mathbf{x}_i \mid y_j, \theta_j)$$

This is mathematically equivalent but numrically more stable and easier to optimize

22.9.2 Gaussian Naïve Bayes

When features are continuos and we assume they follow a Gaussian distribution:

- For each class y_j and feature x_i, we estimate:

$$\mu_{ij} = \frac{1}{N_j} \sum_{k:y_k=y_j} x_{ki}$$

$$\sigma_{ij}^2 = \frac{1}{N_j} \sum_{k:y_k=y_j} (x_{ki} - \mu_{ij})^2$$

where N_j is the number of training samples in class y_j

- The likelihood for feature x_i given class y_j is:

$$p(x_i \mid y_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} exp(-\frac{(x_i - \mu_{ij})^2}{2\sigma_{ij}^2})$$

- Under the independence assumption:

$$p(\mathbf{x} \mid y_j) = \prod_{i=1}^{d} p(x_i \mid y_i)$$

Note. The Gaussian assumption is reasonable for many real-world problems, but it's important to verify whether your data actually follows this distribution. For non-Gaussian data, other parametric models (e.g., multinomial for discrete features) or non-parametric methods may be more appropriate.

22.10 Summary: From Theory to Practice

Bayesian Classification Pipeline:

1. **Theoretical Foundation**: Bayes' theorem provides the optimal decision rule
2. **Practical Challenge**: We don't have access to true probability distributions
3. **Naïve Assumption**: Fit parametric models (e.g., Gaussian) to estimate likelihoods
4. **Parameter Estimation**: Use MLE to learn model parameters from training data
5. **Smoothing**: Apply techniques like Laplace smoothing to handle zero probabilities

23 MLE in Naïve Bayes: Practical Implementation

23.1 MLE for Discrete (Categorical) Features

Suppose you're training Naïve Bayes classifier with **categorical features**. For each class y, you estimate:

- the **prior** $P(y)$ and the **likelihoods** $P(x_i \mid y)$

23.1.1 (a) Prior via MLE

Definizione 23.1 (Prior Estimation)

$$P(y) = \frac{\text{number of samples with class } y}{\text{total number of sample}}$$

Observation. This is simply the relative frequency of class y in the training data. If we have 100 samples and 60 belong to class y_1, then $P(y_1) = 0.6$.

23.1.2 (b) Likelihood via MLE

For categorical features (e.g., words in text classification):

Definizione 23.2 (Likelihood Estimation for Categorical Features)

If x_i is a categorical feature (e.g., word):

$$P(x_i = v_k \mid y) = \frac{\text{count of } (x_i = v_k, y)}{\text{count of } y}$$

where v_k is a specific value (category) that feature x_i can take.

Observation. This is the **maximum likelihood estimate** of the conditional probability $P(x_i = v_k \mid y)$. You're simply estimating probabilities by **relative frequencies**: the parameter values that **maximise** the likelihood of your training data.

Esempio 23.3 (Word Frequency in Spam detection)

If we're classifying emails as spam or ham, and the word "free" appears in 40 out of 60 spam emails, then:

$$P(\text{free} = \text{Yes} \mid \text{Spam}) = \frac{40}{60} = 0.667$$

23.2 MLE for Continuos Features: Gaussian Model

If features are continuos and you assume a Gaussian likelihood:

$$P(x_i \mid y) = \mathcal{N}(x_i; \mu_{iy}, \sigma_{iy}^2) \quad \Rightarrow \quad P(x_i \mid y) = \frac{1}{\sqrt{2\pi\sigma_{iy}^2}} exp(-\frac{(x_i - \mu_i y)^2}{2\sigma_{iy}^2})$$

Then MLE gives:

Definizione 23.4 (Gaussian MLE Parameters)

$$\hat{\mu_i y} = \frac{1}{N_y} \sum_{j:y_j = y} x_{ij}$$

$$\hat{\sigma^2_{iy}} = \frac{1}{N_y} \sum_{j:y_j=y} (x_{ij} - \hat{\mu_i y})^2$$

where N_y is the number of samples in class y.

Key Insight

There are just the **sample mean and variance** of that feature within each class.

23.3 Complete Example: Fruit Classification

Let's work through a complete example of Gaussian Naïve Bayes with continuos features.

Esempio 23.5 (Classifying Fruits by Weight)

We have a dataset of fruits with their weights (in grams):

We want to classify a new fruit with weight $x = 158g$.

23.3.1 Step 1: Estimate Gaussian Parameters (MLE)

For Apples:

Fruit	Weight (g)
Apple	150
Apple	160
Apple	155
Orange	180
Orange	190
Orange	185

Tabella 1: Training data for fruit classification

$$\mu_{\text{Apple}} = \frac{150 + 160 + 155}{3} = \frac{465}{3} = 155 \tag{1}$$

$$\sigma^2_{\text{Apple}} = \frac{(150 - 155)^2 + (160 - 155)^2 + (155 - 155)^2}{3} \tag{2}$$

$$= \frac{25 + 25 + 0}{3} = \frac{50}{3} \approx 16.67 \tag{3}$$

For Oranges:

$$\mu_{\text{Orange}} = \frac{180 + 190 + 185}{3} = \frac{555}{3} = 185 \tag{4}$$

$$\sigma^2_{\text{Orange}} = \frac{(180 - 185)^2 + (190 - 185)^2 + (185 - 185)^2}{3} \tag{5}$$

$$= \frac{25 + 25 + 0}{3} = \frac{50}{3} \approx 16.67 \tag{6}$$

Observation. Notice that both classes have the same variance in this example. This is coincidental and won't always be the case in real data.

23.3.2 Step 2: Compute the Likelihood for a New Sample

New fruit weight: $x = 158$ g.

Using the Gaussian likelihood formula:

$$P(x|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x - \mu_y)^2}{2\sigma_y^2}\right)$$

For Apple:

$$P(158|\text{Apple}) = \frac{1}{\sqrt{2\pi \cdot 16.67}} \exp\left(-\frac{(158 - 155)^2}{2 \cdot 16.67}\right) \tag{7}$$

$$= \frac{1}{\sqrt{104.67}} \exp\left(-\frac{9}{33.34}\right) \tag{8}$$

$$\approx 0.097 \cdot \exp(-0.27) \tag{9}$$

$$\approx 0.097 \cdot 0.763 \tag{10}$$

$$\approx 0.072 \tag{11}$$

For Orange:

$$P(158|\text{Orange}) = \frac{1}{\sqrt{2\pi \cdot 16.67}} \exp\left(-\frac{(158 - 185)^2}{2 \cdot 16.67}\right) \tag{12}$$

$$= \frac{1}{\sqrt{104.67}} \exp\left(-\frac{729}{33.34}\right) \tag{13}$$

$$\approx 0.097 \cdot \exp(-21.87) \tag{14}$$

$$\approx 0.097 \cdot 3.0 \times 10^{-10} \tag{15}$$

$$\approx 1.6 \times 10^{-11} \tag{16}$$

Observation. The likelihood for Apple is much higher than for Orange. This makes intuitive sense: 158g is much closer to the Apple mean (155g) than to the Orange mean (185g).

23.3.3 Step 3: Multiply by Class Prior

Assume equal priors (we have 3 apples and 3 oranges):

$$P(\text{Apple}) = P(\text{Orange}) = 0.5$$

Computing the posterior (proportional to likelihood $\times$ prior):

$$P(\text{Apple}|158) \propto P(158|\text{Apple}) \cdot P(\text{Apple}) \tag{17}$$
$$= 0.072 \cdot 0.5 \tag{18}$$
$$\approx 0.036 \tag{19}$$

$$P(\text{Orange}|158) \propto P(158|\text{Orange}) \cdot P(\text{Orange}) \tag{20}$$
$$= 1.6 \times 10^{-11} \cdot 0.5 \tag{21}$$
$$\approx 8 \times 10^{-12} \tag{22}$$

Prediction: Apple, because posterior probability is much higher.

$$P(\text{Apple} \mid 158) \approx 0.036 \gg P(Orange \mid 158) \approx 8 \times 10^{-12}$$

24 Nonparametric Classifiers

24.0.1 Introduction to Nonparametric Methods

So far, we've discussed parametric approaches (like Gaussian Naive Bayes) that assume a specific form for the probability distribution. Now we turn to **nonparametric methods**.

Definizione 24.1 (Nonparametric Methods)

Given a set of examples, model the density function of the data **without making any assumptions about the form of the distribution** (e.g., Gaussian distribution).

- Estimate the probability density function **directly from the samples**
- No predeterined functional form (e.g., no assumption of Gaussian, exponential, etc.)

Observation. The term "nonparametric" is somewhat misleading - these methods often have parameters (like bin width in histograms or bandwidth in kernel density estimation), but they don't assume a specific parametric form for the underlying distribution.

Key Idea

The simplest form of nonparametric density estimation is the **histogram**.

25 Histogram-Based Density Estimation

The histogram is the most intuitive nonparametric density estimator.

25.0.1 How Histogram Work

Definizione 25.1 (Histogram Density Estimator)

Dividing the sample space into number of bins and approximate the density at the **center of each bin** by the **fraction** of points in the training data that fall into the corresponding bin:

$$p_H(x) = \frac{1}{N} \cdot \frac{\#\text{ of } x^{(k)} \text{ in same bin as } x}{\text{width of bin}}$$

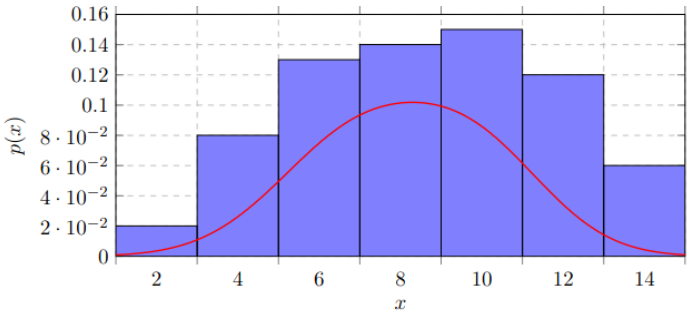
where N is the total number of training samples.

Observation. The histogram essentially counts how many data points fall into each bin and normalizes by the bin width and total number of samples to create a probability density estimate.

25.0.2 Hisogram Parameters

The histogram requires two "parameters" to be defined:

1. **Bin width**: Determines the resolution of the density estimate
2. **Starting position** of the first bin: Determines where the bins are anchored



25.1 Drawbacks of Histograms

Despite their simplicity, histograms have several significant limitations:

1. **Dependence on starting position**: The density estimate depends on the starting position of the bins
   - Different starting positions can produce different density estimates
   - For multivariate data, the density estimate is also affected by the **orientation of the bins**
2. **Discontinuities**: The discontinuities of the estimate are not due to the underlying density they are only an artifact of the chosen bin locations

- These discontinuities make it very difficult to grasp the **structure** of the data
- The true underlying distribution is typically smooth, but histograms create artificial jumps

3. **Curse of dimensionality**: A much more serious problem is the **curse of dimensionality**, since the number of bins grows exponentially with the number of dimensions
   - In high dimensions we would require a very large number of examples or else most of the bins woulde be empty
   - With d dimensions and k bins per dimensio, we need k^d total bins

The curse of dimensionality

As the number of features or dimensions increases, the amount of data required to effectively cover the feature space increases **eponentially**

25.2 The curse of Dimensionality

Definizione 25.2 (Curse of Dimensionality)

The curse of dimensionality refers to the challenges and issues that arise when working with high-dimensional data in machine learning.

As the number of features or dimensions in a dataset increases, the amount of data required to effectively cover the feature space also increases eponentially.

25.2.1 Why it matters

- **Exponential growth**: With d dimensions, if each dimension is divided into k bins, we need k^d bins total.
- **Data sparsity**: In high dimensions, data points become increasingly sparse
  - Most bins will be empty unless we have an enormous amount of data
  - The distance between points increases, making it harder to find meaningful patterns
- **Sample size requirements**: To maintain the same density of samples, we need exponentially more data as dimensions increase

Esempio 25.3 (Exponential Growth of Bins)

- 2D: 10 bins per dimension $\rightarrow$ 100 bins total
- 3D: 10 bins per dimension $\rightarrow$ 1,000 bins total
- 10D: 10 bins per dimension $\rightarrow$ 10,000,000,000 bins total

25.2.2 Mitigatin the Curse of Dimensionality

To mitigate the curse of dimensionality, various techniques are employed:

- **Feature selection**: Choose only the most relevant features
- **Dimensionality reduction**: Techniques like Principal Component Analysis (PCA) reduce the number of dimensions while preserving important characteristics
- **Regularization methods**: Add constraints to prevent overfitting in high dimensions
- **Domain knowledge**: Use expert knowledge to identify which features are truly important

Practical Implication

In practice, nonparametric methds like histograms work well for 1D or 2D data, but become impractical for high-dimensional data. This motivates the development of more sophisticated techinques.

25.3 Probability Density: Mathematical Foundation

Before we proceed with more advanced nonparametric techinques, let's return to the basic definition of probability to get a solid idea of what we are trying to accomplish.

Definizione 25.4 (Probability in a Region)

The probability that a vector $\mathbf{x}$, drawn from a distribution $P(\mathbf{x})$, will fall in a region $\mathcal{R}$ of the sample space is:

$$P = \int_{\mathbf{x} \in \mathcal{R}} P(\mathbf{x}')\, dx$$

where:

- $\mathbf{x}$ is a particular vector we are asking about ("drawn from the distribution")
- $\mathbf{x}'$ is a dummy variable used to sum/integrate over all vectors in $\mathcal{R}$.
- $\mathcal{R}$ is a region in the feature space

Observation. This integral formuulation is the founation for all density estimation methods. The key insight is that if we can estimate the probability is small regions around each point, we can reconstruct the entire probability density function.

25.3.1 Key concepts

- **Probability density function** $P(\mathbf{x})$: Describe how probability is distributed over the feature space

- **Integration over a region**: The probability of falling in region $\mathcal{R}$ is the integral of the density over that region
- **Dummy variable** $\mathbf{x}'$: Used for integration; represent all possible points in the region
- **Normalization**: The integral over the entire space must equal 1:

$$\int_{\text{all space}} P(\mathbf{x}')\, dx' = 1$$

Note. For discrete distributions, the integral become a sum:

$$P = \sum_{\mathbf{x}' \in \mathcal{R}} P(\mathbf{x}')$$

This is the discrete analog of the continuos case and is what we use when working with categorical data.

25.4 Binomial Distribution Approach

Now let's develop a more rigorous approach to nonparametric density estimation based on probability theory.

25.4.1 Setup

Suppose now that N vectors $\{\mathbf{x}(1), \mathbf{x}(2), \ldots, \mathbf{x}(N)\}$ are drawn **independently and identically distributed (i.i.d.)** from the distribution. The probability that k of these N vectors fall in $\mathcal{R}$ is given by the binomial distribution:

Definizione 25.5 (Binomial distribution)

$$P(k) = \binom{N}{k} p^k (1-p)^{N-k}$$

where:

$$\binom{N}{k} = \frac{N!}{k!(N-k)!}$$

and the expected value for k is:

$$\mathbb{E}[k] = NP$$

Observation. **Recall**: The expectation is the average value we would get if we repeated drawing samples many times.

The binomial distribution tells us how likely it is to observe exactly k points in region $\mathcal{R}$ when we draw N samples, given that each point has probability P of falling in $\mathcal{R}$.

25.4.2 Mean and Variance of the Ratio

The mean and variance of the ratio k/N are:

$$\mathbb{E}[\frac{k}{N}] = P$$

$$\text{Var } [\frac{k}{N}] = \mathbb{E}[(\frac{k}{N} - P)^2] = \frac{P(1-P)}{N}$$

Observation. Therefore, as $N \to \infty$, the distribution becomes **sharper**, the variance gets **smaller**, thus we can obtain a **better estimate** of the probability P from the mean fraction of the points that fall within $\mathcal{R}$:

$$P \cong \frac{k}{N}$$

This is the law of large numbers in action: with more samples, our estimate becomes more accurate.

25.5 Density Estimation Formula

25.5.1 Small Region Assumption

On the other hand, if we assume that $\mathcal{R}$ is **so small** that $P(\mathbf{x})$ **does not vary** appreciably within it, then:

$$\int_{\mathcal{R}} P(\mathbf{x}')\, dx' \cong P(\mathbf{x})V$$

where V is the **volume enclosed** by region $\mathcal{R}$.

Observation. This approximation assumes that the probability density is approximately constant within the small region $\mathcal{R}$, so we can pull $P(\mathbf{x})$ out of the integral, leaving just the volume V.

25.5.2 Combining the Results

Merging with the previous result, we obtain:

$$P = \int_{\mathcal{R}} p(\mathbf{x}')\, dx' \cong P(\mathbf{x})V$$

where V is the **volume enclosed** by region $\mathcal{R}$.

Obseravation. This approximation assumes that the probability density it approximately constant withint the small region $\mathcal{R}$, so we can pull $P(\mathbf{x})$ out of the integral, leaving just the volume V.

25.5.3 Combining the Results

Merging with the previous result, we obtain:

$$P = \int_{\mathcal{R}} p(\mathbf{x}')\, dx' \cong p(\mathbf{x})V$$

$$P \cong \frac{k}{N}$$

Therefore:

$$p(\mathbf{x}) \cong \frac{k}{NV}$$

Key Result

This estimate becomes more accurate as we **increase the number of sample points** N and **shrink the volume** V

25.6 The Trade-off Problem

In practice, the value of N (the total number of examples) is fixed.

25.6.1 The dilemma

- In order to improve the accuracy of the estimate $P(\mathbf{x})$ we could let V to approach zero, but then the region $\mathcal{R}$ would become so small that it would enclose no examples
- This means that in practice, we will have to find a **compromise value of the volume** V:
  - **Large enough** to include enough examples withing $\mathcal{R}$.
  - **Small enough** to support the assumption that $P(\mathbf{x})$ is constant within $\mathcal{R}$

The fundamental Trade-off

- Too large $V \rightarrow$ Poor approximation (density varies within region)
- Too small $V \rightarrow$ Too few samples (high variance in estimate)

25.7 General Nonparametric Density Estimation Formula

Teorema 25.6 (General Expression for Nonparametric Density Estimation)

$$P(\mathbf{x}) \cong \frac{K}{NV}$$

where:

- V is the volume surrounding $\mathbf{x}$.
- N is the total number of examples
- k is the number of examples inside V

25.8 Two Approaches to Nonparametric Density Estimation

When applying this result to practical density estimation problems, two basic approaches can be adopted:

1. **Fix V and determine k from the data**
   - This leads to **kernel density estimation (KDE)**, e.g., Parzen window
   - We define a fixed-size region around each point and count how many samples fall within it
2. **Fix k and determine V from the data**
   - This gives rise to the **k-nearest-neighbor (k-NN) approach**
   - We find the k nearest neighbors and measure the volume needed to enclose them

Convergence Property

Both k-NN and KDE converge to the true probability density as $N \to \infty$, provided that:

- V shrinks with N.
- k grows with N appropriately

25.8.1 Comparison of Approaches

Observation. **KDE** produces smoother estimates but may miss structure in sparse regions, while **k-NN** adapts to local density but can produce less smooth estimates. The choice depends on the specific application and data characteristics

Kernel Density Estimation (KDE)	**k-Nearest Neighbor (k-NN)**
Fix volume V, Count samples k in fixed region	Fix number of neighbors k. Adjust volume V to include k sam
Smooth density estimate	Adaptive to local density
Can be zero in sparse region	Always use k neighbors
Examples: Parzen window, Gaussian kernel	Examples: k-NN classifier, k-NN regressor

25.8.2 Choosing Parameters

- **For KDE**: Choose bandwidth (related to V)
  - Too small: noisy, overfitting
  - Too large: oversmoothing, missing details
- **For k-NN**: Choose k:
  - Too small: sensitive to noise, high variance
  - Too large: oversmoothing, high bias

Note. Both methods require careful tuning of their parameters (bandwidth for KDE, k for k-NN). Corss-validation is commonly used to select these parameters in practice.

26 k-Nearest Neighbors (k-NN)

26.1 Visual Comparison: k-NN vs. Parzen Windows

Key difference:

- **Parzen Window**: Fixed volume V (radius R), variable k
- **k-NN**: Fixed k (number of neighbors), variable volume V (radius R)

Observation. In the visual comparison:

- **Parzen Window**: Uses a fixed-size circle (radius R) around the query point (yellow cross). The number of points inside varies depending on local density
- **k-NN (e.g., 3-NN)**: The circle grows or shrinks to always include exactly $k = 3$ nearest neighbors. In dense regions, the circle is small; in sparse regions, it's large

26.2 Parzen Window: Practical Consideration

When to use Parzen Windows

Parzen window is great for **low-dimensional data** (2-3 features, maybe up to 5)

- In higher dimension, the method becomes **computationally expensive**, and the density estimates are **unreliable** because of the curse of dimensionality
- For this reason, in this year, we focus on other methods better suited for high-dimensional data

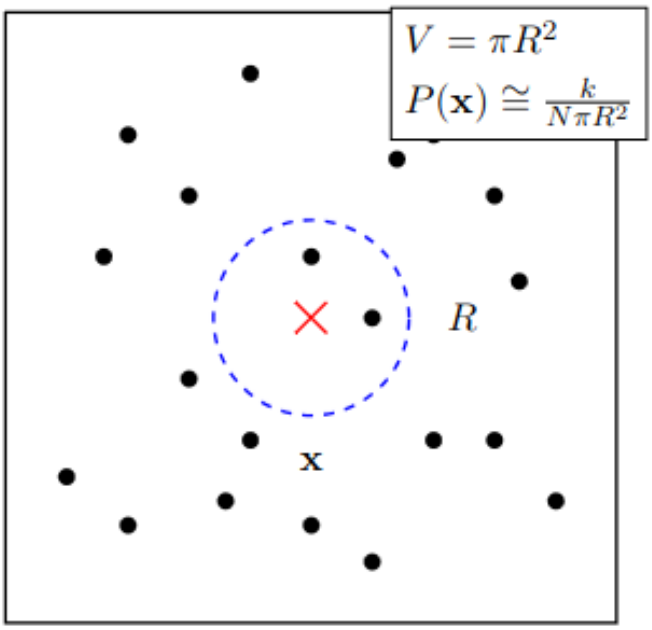
Note. While Parzen windows provide a smooth density estimates, they suffer from the curse of dimensionality. As dimension increase, the fixed window size becomes either too large (over-smoothing) or too small (too few samples), making the method impractical for modern high-dimensional datasets.

26.3 k-Nearest Neighbors Algorithm

The k-NN algorithm is one of the simplest and most intuitive machine learning algorithms

26.3.1 Basic principle

The volume is grown surrounding the estimation point $\mathbf{x}$, until it enclose a total of k data points.



Definizione 26.1 (k-NN Density Estimation)

$$P(\mathbf{x}) \cong \frac{k}{NV}$$

where:

- V is the volume surrounding $\mathbf{x}$.
- N is the total number of examples
- k is the number of examples inside V (fixed)

26.4 k-NN for Classification

26.4.1 1-Nearest Neighbor (1-NN)

Given an unknown point $\mathbf{x}^*$, we consider $s_i \in \{s_1, \ldots, s_N\} = S$ the nearest point (NN) of the point $\mathbf{x}^*$ if:

$$d(\mathbf{x}^*, s_i) = \min_l d(\mathbf{x}^*, s_l), \quad l = 1 \ldots N$$

where $d(.)$ is a **distance measure** defined on the feature space.

Observation. If you interpret each point of set S as a prototype of a class, you can then see the equation above as a rule of classification **1-NN** associating the sample $\mathbf{x}$ to the class j to which the point s_i belongs.

Esempio 26.2 (Common Distance Measures)

- **Euclidean distance**: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{d}(x_i - y_i)^2}$
- **Manhattan distance**: $d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{d} \mid x_i - y_i \mid$
- **Minkowski distance**: $d(\mathbf{x}, \mathbf{y}) = (\sum_{i=1}^{d} \mid x_i - y_i \mid^P)^{\frac{1}{P}}$

26.4.2 Generalizing to k-NN

Rule 1-NN can be generalised to define a k-**NN rule**, which consists of determining the k points belonging to the set S closest to the observation $\mathbf{x}$.

Definizione 26.3 (k-NN Classification Rule)

The classification rule k-NN associates observation $\mathbf{x}$ with class i having the **largest number of elements** among the nearest k. We call $U(\mathbf{x})$ the set of k points closer to $\mathbf{x}$.

26.4.3 Majority Voting

For example, with odd k and two classes, y_1 and y_2, the decision rule can choose the class with the most samples in $U(\mathbf{x})$. This is called **majority voting**.

Majority Voting

Choose the class that appears most frequently among the k nearest neighbors.

Esempio 26.4 (3-NN Classification)

Suppose we have $k = 3$ and two classes (red and blue):

- Query point $\mathbf{x}^*$ has 3 nearest neighbors: 2 red, 1 blue
- Majority voting: $\mathbf{x}^*$ is classified as **red**

Rule of Thumb

- Start with $k = \sqrt{N}$ where N is the number of training samples
- Use cross-validation to find the optimal k
- For binary classification, use odd k to avoid ties

Advantages	Disadvantages
• Simple and intuitive	• Computationally expensive at prediction time
• No training phase (lazy learning)	• Requires storing all training data
• Naturally handles multi-class problems	• Sensitive to irrelevant features
• Non-parametric (no assumptions about data distribution)	• Sensitive to the scale of features
• Can capture complex decision boundaries	• Curse of dimensionality
• Easy to update with new data	• Choosing optimal k can be difficult

Tabella 2: Advantages and disadvantages of k-NN

26.5 Advantages and Disadvantages of k-NN

26.6 Practical Considerations

26.6.1 Feature Scaling

Observation. k-NN is **highly sensitive to feature scaling** because it relies on distance calculations. Features with larger ranges will dominate the distance metrics.

Solution: Always normalize or standardize features before applying k-NN:

- **Min-Max Normalization**: Scale to [0,1]: $x' = \frac{x - \min(x)}{\max(x) - \min(x)}$
- **Standardization**: Scale to mean 0, std 1: $x' = \frac{x - \mu}{\sigma}$

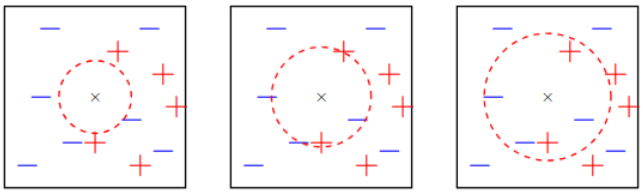
26.6.2 Computational Complexity

- **Training time**: $O(1)$ (no training, just store data)
- **Prediction time**: $O(Nd)$ where N is training set size, d is dimensionality
- For large datasets, use approximate nearest neighbor algorithms (e.g., KD-Trees, Ball trees LSH)

Note. k-NN is called a **lazy learner** or **instance-based learner** because it doesn't learn a model during training. Instead, it memorizes the training data and performs all computation at prediction time. This makes training fast but prediction slow, especially for large datasets.

26.7 Distance Metrics in k-NN

To calculate the distance, we usually use **Euclidean distance**.



(m) 1-nearest neighbor (m) 2-nearest neighbor (m) 3-nearest neighbor

Definizione 26.5 (Euclidean Distance Formula)

Given two points $\mathbf{p} = (p_1, p_2, \ldots, p_n)$ and $\mathbf{q} = (q_1, q_2, \ldots, q_n)$:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \ldots + (q_n - p_n)^2} = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

Observation. The visual shows how the radius grows as k increases:

- **1-NN**: Small circle, includes only the closest neighbor
- **2-NN**: Medium circle, includes 2 closest neighbors
- **3-NN**: Larger circle, includes 3 closest neighbors

The decision boundary depends on which class dominates within the circle.

26.8 Weighted k-NN

26.8.1 The problem with Equal Weights

All the closest neighbors have the same weights, meaning that they are equally important to make a decision.

Issue: Therefore, the algorithm is very senstivie of the number of k.

26.8.2 Solution: Distance-Based Weighting

This can be reduced by **weighing the contribution of neighbors based on distance** $w = 1/d(\mathbf{x}^*, s_i)$, such that the **closet** neighbor would have more **importance**.

Definizione 26.6 (Weighted k-NN)

Instead of simple majority voting, each neighbor contributes with weight:

$$w_i = \frac{1}{d(\mathbf{x}^*, s_i)}$$

The class with the highest weighted sum is chosen:

$$\text{class}(\mathbf{x}^*) = \arg\max_c \sum_{i \in U(\mathbf{x}^*)} w_i \cdot \mathbb{1}(y_i = c)$$

where $\mathbb{1}(y_i = c)$ is 1 if neighbor i belongs to class c, and 0 otherwise.

Esempio 26.7 (Weighted vs. Unweighted k-NN)

Consider $k = 3$ with neighbors at distance 1,2 and 5:

- **Unweighted**: Each neighbor gets weight 1
- **Wighted**: Neighbors get weights $w_1 = 1/1 = 1.0$, $w_2 = 1/2 = 0.5$, $w_3 = 1/5 = 0.2$.

The closest neighbor has much more influence in the wighted version.

26.9 Importance of Feature Scaling (Revisited)

26.9.1 Why k is Sensitive

The choice of k is important because:

- if k is too **small**, the approach is **sensitive to noise**
- if k is too **large**, the neighborhood can include examples belongin to **other classes**

Critical Requirement

To operate correctly the attributes must have the **same value scale** and must therefore be **normalized** in the preprocessing phase.

Observation. Without normalization, features with larger ranges will dominate the distance calculation, making features with smaller ranges essentially irrelevant to the classification decision.

26.10 Mahalanobis Distance

26.10.1 When Euclidean Distance is not Enought

Normalizing the scale may not be sufficient when there are different data distributions.

In such cases instead of using Euclidean distance, one can also try using **Mahalanobis distance**

26.10.2 Definition

The Mahalanobis distance is a measure of the distance between **a point and a distribution**. It's a generalized form of the Euclidean distance that accounts for **correlations between variables** and **different variances** along different axes.

Definizione 26.8 (Mahalanobis Distance formula)

$$D = \sqrt{(\mathbf{x} - \mu)^{T\Sigma^{-1}}(\mathbf{x} - \mu)}$$

where:

- $\mathbf{x}$ is the n-dimensional vector representing the point's coordinates
- μ is the n-dimensional vector representing the mean of the distribution
- Σ is the $n \times n$ covariance matrix of the distribution
- Σ^{-1} is the inverse of the covariance matrix

Observation. When the covariance matrix is the identity matrix (i.e., features are uncorrelated and have unit variance), the Mahalanobis distance reduces to the Euclidean distance.

26.10.3 When to Use Mahalanobis distance

- **Correlated Features**: Changes in one feature are associated with changes in antoher.
- **Unequal Variances**: MD normalizes the distance based on the variances of the features, preventing features with larger variances from dominating the distance calculation
- **Outlier Detection**: Outliers often have a large MD from the rest of the data points, especially if they deviate from the normal covariance pattern.
- **In high dimensional spaces**: ED can be problematic (*curse of dimensionality*). Incorporating information about the covariance structure, can mitigate this issue to some extent.

Esempio 26.9 (Mahalanobis Distance in Practice)

Consider a 2D dataset where:

- Feature 1 has variance 1
- Feature 2 has variance 100

Euclidean Distance	Mahalanobi
Assumes features are independent	Accounts for correlatio
Treats all directions equally	Accounts for different varia
Sensitive to feature scaling	Automatically account
Simple and fast to compute	More computationally expensiv
Works well when features are uncorrelated and have similar variance	Better when features are correla
Distance from point to point	Distance from po

- Features are positively correlated

Euclidean distance would be dominated by Feature 2 due to its large variance.

Mahalanobis distance would:

- Normalize for the different variances
- Account for the correlation structure
- Provide a more meaningful distance measure

Note. Computing Mahalanobis distance requires estimating the covariance matrix Σ, which can be challenging when:

- The number of features is large relative to the number of samples
- The covariance matrix is singular (not invertible)
- There is insufficient data to reliably estimate correlations

In such cases, regularization techniques or dimensionality reduction may be needed before applying Mahalanobis distance.

26.10.4 Visual Example: Euclidean vs. Mahalanobis Distance

Esempio 26.10 (Comparing Distance Metrics)

Consider a datsaset centroid at $(\overline{x}, \overline{y}) = (3.1, 3.0)$ and two data points:

- **Data point 1**: (5,5) (yellow/orange point)
- **Data point 2**: (5,1) (blue point)

Using Euclidean distance:

Distance from centroid to Data point 1:

$$d_1 = \sqrt{(5-3.1)^2 + (5-3.0)^2} = \sqrt{1.9^2 + 2.0^2} = \sqrt{3.61 + 4.00} = 2.76$$

Distance from centroid to Data point 2:

$$d_2 = \sqrt{(5-3.1)^2 + (1-3.0)^2} = \sqrt{1.9^2 + (-2.0)^2} = \sqrt{3.61 + 4.00} = 2.76$$

Based on Euclidean distance, both points have the same distance to the centroid!

Observation. However, looking at the scatter plot, we can see that the green points (the cluster) are elongated along a diagonal direction. Data point 1 lies along this diagonal trend, while Data point 2 deviates significantly from it.

Euclidean distance treats all directions equally and doesn't account for the correlation structure of the data.

Esempio 26.11 (Using Mahalanobis Distance)

When we apply Mahalanobis distance to the same scenario:

- $\mathbf{MD}_1 = \mathbf{2.26}$ (Data point 1 to centroid)
- $\mathbf{MD}_2 = \mathbf{6.45}$ (Data point 2 to centroid)

Why is $\mathbf{MD}_1$ so different from $\mathbf{MD}_2$?

Because MD takes into account the **correlation** in the data

Interpretation:

- Data point 1 follows the natural diagonal trend of the cluster $\rightarrow$ smaller Mahalanobis distance
- Data point 2 deviates from the correlation pattern $\rightarrow$ larger Mahalanobis distance

Note. This example illustrates why Mahalnobis distance is superior when features are correlated:

- **Euclidean distance**: Treats both points as equally distant (2.76)
- **Mahalanobis distance**: Correctly identifies that point 2 is much farther from the distribution (6.45 vs 2.26)

Mahalanobis distance accounts for the sahep and orientation of the data distribution, not just row coordinate distances.

26.11 How to select k in k-NN

Choosing the right value of k is crucial for k-NN performance. Here are practical strategies:

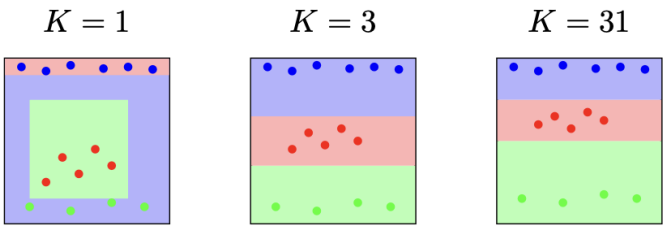
26.11.1 Common Heuristics

- **Common values**: Often 3, 5, 7
- **Choose an odd number** to avoid ties (*undetermined cases*)
- **Use your training and validation data** to decide k:
  - Change k from $k = 1$ until the validation performance decreases
  - Pay attention not to **underfit** and **overfit** the data
  - Finilize the decision of k and use the same k to classify the test data
- **A typical choice** is $k \approx \sqrt{N}$ where N is the number of training samples

Validation-Based Selection

The most reliable method is to use cross-validation:

1. Try different values of k (e.g., $1, 3, 5, 7, 9, 11, \ldots$)
2. Evaluate performance on validation set for each k
3. Select the k that gives the best validation performance
4. Use this optimal k on the test set



What is the role of k? How does it relate to overfitting and underfitting?

- $k = 1$ (left): Very complex, kagged decision boundary
  - High variance, low bias
  - **Overfitting**: Captures noise in the training data
  - Sensitive to outliers
- $k = 3$ (middle): Moderate complexity, smoother boundaries
  - Balanced bias-variance trade-off
  - **Underfitting**: May miss important patterns
  - Too much smoothing

Observation: As k increases:

- Decision boundaries become smoother and simpler
- The model becomes less sensitive to individual data point
- Risk shifts from overfitting (small k) to underfitting (large k)

26.11.2 Handling Ties (Undetermined Cases)

If there are undetermined cases (ties), there are some alternatives:

1. **Arbitrary choice**: Randomly select one of the tied classes
2. **Nearest average sample**: Assign $\mathbf{x}$ to the class (among those 'tie') that has the nearest average sample (calculated between the k_i sample)
3. **Least distance**: Assign $\mathbf{x}$ to the class that has the k_i samples with least distance from it

Esempio 26.12 (Handling a Tie)

Suppose $k = 4$ and we have:

- 2 neighbors from class A at distances: 1.5, 2.0
- 2 neighbors from class B at distances: 1.2, 3.5

Solutions:

- **Arbitrary**: Randomly choose A or B
- **Nearest average**:
  - Class A average: (1.5 + 2.0) / 2 = 1.75
  - Class B average: (1.2 + 3.5) / 2 = 2.35
  - Choose class A (smaller average distance)
- **Least distance**: Choose class B (has the closest neighbor at 1.2)

Note. The best strategy for handling ties depends on your specific problem:

- **Nearest average**: Good when you want to consider all neighbors equally
- **Least distance**: Good when you believe the closest neighbor is most important
- **Best practice**: Use odd k to aovid ties altogether in binary classification

26.12 k-NN: Pros and Cons

26.12.1 Advantages (Pros)

- **Almost no assumptions about the data**
  - Non-parametric method
  - Doesn't assume any particular distribution
  - Works with any data strucutre
- **Non-parametric**
  - No model parameters to learn
  - Flexible and adaptive to data

- **There is no training phase, cost is 0**
  - Instant "training" - just store the data
  - Easy to update with new data
  - No convergence issue
- **Allow us to construct non-linear class boundaries and are therefore more flexible**
  - Can capture complex decision boundaries
  - Not limited to linear seprability
  - Adapts to local data structure

k-NN gives locally defined decision boudaries between classes

Unlike global methods (e.g., linear classifier), k-NN creates decision boundaries that adapt to the local structure of the data, allowing for complex, non-linear separations.

Obseravation. The figure shows how k-NN can create complex, non-linear decision boundaries (shown in red) that follow the natural clustering of the data. The decision boundary is not a simple line or hyperplane, but rather a complex curve that adapts to the local density and distribution of each class (label 1,2 and 3)

On the right, we see the resulting decision regions where each color represents the area classified as a particular label. The boundaries are highly non-linear and follow the intricate patterns in the data

26.12.2 Disadvantages (Cons)

- **Need to handle missing data**
  - Cannot compute distances with missing values
  - Requires imputation or special handling
- **The distance metric should be chosen carefully**
  - Different metrics can give very different results
  - Must consider feature correlations and scales
  - May need domain knowledge
- **Sensitive to class outliers** (e.g., mislabeled training instances)
  - Outliers can create incorrect local decision boundaries
  - No mechanism to detect or remove bad training examples
- **Sensitive to many irrelevant attributes** (which influence the distance)
  - All features contribute equally to distance
  - Irrelevant features add noise
  - Feature selection is crucial

- **Computationally expensive**:
  - **Poor scalability**: For each new example, we have to perform as many comparisons as there are labeled data
  - **It is necessary to store all data points**: Therefore, the method is more appropriate when the number of samples N is low
  - Memory requirements grow linearly with dataset size
  - Prediction time is $O(Nd)$ where N is training size, d is dimensionality

Scalability issue

k-NN does not scale well to large datasets. For a dataset with millions of samples, making a single prediction requires computing distances to all training points, which becomes prohibitively expensive

Note. The main trade-off with k-NN is between flexibility and computational cost:

- **Pros**: Simple, flexible, no training, handles non-linear boundaries
- **Cons**: Slow predictions, high memory, sensitive to irrelevant features and outliers

k-NN works best for:

- Small to medium-sized datasets
- Problems where decision boundaries are complex and non-linear
- Applications where training time is critical but prediction time is less important
- Situations where new data arrives frequently (easy to update)

26.13 How k-NN Was Born: PDF Estimation

k-NN has a theoretical foundation in probability density function (PDF) estimation. Let's explore this connection.

26.13.1 Going back to PDF Estimation

Recall from our earlier discussion of density estimation:

- Instead of fixing V as in the case of kernels (Parzen windows), we **fix the value of** k and enlarge the radius of the sphere to include k samples
- Let's see the **a-priori probability**:
  - Given a class y_i, the frequency of occurence of samples N_j of class j is generally simply measured in relation to the total number of samples N, i.e.,

$$P(y_j) \equiv \hat{p}(y_j) = \frac{N_j}{N}$$

Observation. This is the empirical estimate of the prior probability: the proportion of training samples that belong to class j.

26.13.2 Density Estimation and Nearest-Neighbor Rule

Consider 2 classes, y_i and y_j containing N_i and N_j samples, $N = N_i + N_j$.

The local density estimation for y_i is calculated as (and similarly for y_j):

$$\hat{p}(\mathbf{x} \mid y_i) = \frac{1}{V}\frac{k_i}{N_i}$$

such that the ratio of $k_i(k_j)$ points to the total $N_i(N_j)$ belonging to the class $y_i(y_j)$ contained in the volume V.

Definizione 26.13 (Deriving k-NN from Bayes Rule)

The Bayes rule says $p(\mathbf{x} \mid y_i)P(y_i) > p(\mathbf{x} \mid y_j)P(y_j)$, then

$$\begin{aligned}
&\hat{p}(\mathbf{x} \mid y_i)\hat{p}(y_i) > \hat{p}(\mathbf{x} \mid y_j)\hat{p}(y_j) \\
\Rightarrow &\frac{1}{V}\frac{k_i}{N_i} \cdot \frac{N_i}{N} > \frac{1}{V}\frac{k_j}{N_j} \cdot \frac{N_j}{N} \\
\Rightarrow &\frac{1}{VN_i}k_iN_i > \frac{1}{VN_j}k_jN_j \\
\Rightarrow &k_i > k_j
\end{aligned}$$

The k-NN Rule

Classify $\mathbf{x}$ to class y_i if $k_i > k_j$, i.e., if there are more neighbors from class i than from class j in the volume V around $\mathbf{x}$.

Observation. This derivation shoes that k-NN is actually implementing a Bayesian decision rule using local density estimates! The key insight is:

- We estimate $p(\mathbf{x} \mid y_i)$ locally using k nearest neighbors
- We estimate $P(y_i)$ using the proportion of class i in the training set
- The Bayes rule simplifies to: choose the class with the most neighbors in the local region

Note. This theoretical foundation explains why k-NN works:

- It's approximating the optimal Bayes clasifier
- It uses local density information to make decision
- The quality of the approximation depends on:
  - Choice of k (too small = noisy estimates, too large = onversmoothing)
  - Density of training data (more data = better estimates)
  - Dimensionality (curse of dimensionality affects density estimation)

As the number of training samples $N \to \infty$ and $k \to \infty$ (but $k/N \to 0$), the k-NN classifier approaches the optimal Bayes classifier.

Parte IV
Decision Trees and Ensemble Methods

27 Introduction to Decision Trees

Decision trees are fundamental machine leraning models that provide both powerful predictive capabilities and high interpretability. They are tree-structured prediction models that make decision by routing data through a hierarchy of test.

27.1 Decision Tree Structure

A decision tree is composed of two types of nodes:

- **Non-terminal nodes**: These nodes have 2 or more children and implement a **routing function**. They perform tests on the input features to decide which branch to follow.
- **Leaf nodes (terminal nodes)**: These nodes have no children and implement a **prediction funciton**. They provide the final output of the model.

Definizione 27.1 (key Structural Properties)

- There are no cycles in the tree structure
- All nodes have at most 1 parent
- There is exactly one node with no parent, called the **root node**
- The tree forms a directed acyclic graph (DAG)

27.2 How decision Trees Work

A decision tree takes an input $x \in \mathcal{X}$ and routes it through its nodes until it reaches a leaf node, where a prediction is made.

The process:

1. The input enters at the root node
2. Each node represents a test on a variable (feature)
3. An edge to a child node represents the outcome of the test
4. The routing continues until a leaf node is reached
5. At the lead, a prediction is made based on the prediction function

27.3 Decision Tree Inference: Non-terminal Nodes

Each non-terminal node can be formally represented as Node(ϕ, t_L, t_R), where:

- $\phi : \mathcal{X} \to \{L, R\}$ is a **routing function**
- t_L is the left child subtree
- t_R is the right child subtree

When an input $\mathbf{x}$ reaches the node:

- If $\phi(\mathbf{x}) = L$, the input goes to the left child t_L
- If $\phi(\mathbf{x}) = R$, the input goes to the right child t_R

Note: We assume binary trees here, but the concept generalizes to tree with more than two children.

27.4 Decision Tree Inference: Leaf Nodes

Each leaf node can be represented as Leaf(h), where:

- $h \in \mathcal{F}_{\text{task}}$ is a **prediction function** (typically a costant)
- Depending on the task
  - For **classification**: $h \in \mathcal{Y}$ (a class label)
  - For **regression**: $h \in \mathbb{R}$ (a continuous value)

Once $\mathbf{x}$ reaches the leaf, the final prediction is given by $h(\mathbf{x})$.

27.5 Formal Decision Tree Function

The complete decision tree function can be defined recursively:

$$f_t(\mathbf{x}) = \begin{cases} h(\mathbf{x}) & \text{if } t = \text{Leaf}(\dots) \\ f_{t_{\phi(\mathbf{x})}}(\mathbf{x}) & \text{if } t = \text{Node}(\phi, \dots) \end{cases}$$

Interpretation:

- If we are at a leaf nodem return the prediction $h(\mathbf{x})$
- If we are at a non-terminal node, apply the routing function $\phi(\mathbf{x})$ to determine which child to visti, then recursively apply the tree function to that child

27.6 Decision Tree Inference Example

Consider a 2D classification problem with classes $\xi \in \{+, \times, \circ, \triangle, \square\}$. We can build a decision tree with routing functions:

$$\phi_1(x, y) = \begin{cases} L & \text{if } y \geq 3 \\ R & \text{else} \end{cases}$$

$$\phi_2(x,y) = \begin{cases} L & \text{if } x \leq 3 \\ R & \text{else} \end{cases}$$

$$\phi_3(x,y) = \begin{cases} L & \text{if } x \leq 2 \\ R & \text{else} \end{cases}$$

$$\phi_4(x,y) = \begin{cases} L & \text{if } x \leq 4 \\ R & \text{else} \end{cases}$$

These routing functions partition the 2D space into axis-aligned rectangular regions, with each region corresponding to a lead node that predicts a specific class.

Parte V
Linear Discriminant Functions and Classiification Methods

28 Linear Discriminant functions

The objective of this lecture is to present methods for learning **linear discriminant** functions of the form:

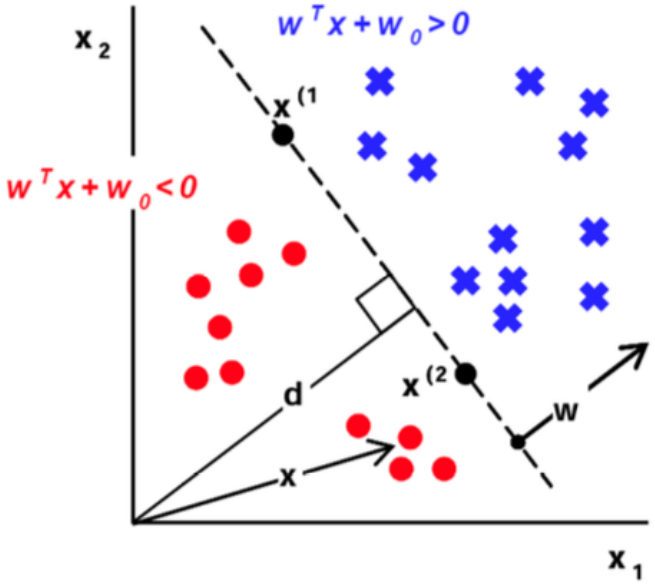
$$g(x) = w^T x + w_0 \Leftrightarrow \begin{cases} g(x) > 0 & x \in \omega_1 \\ g(x) < 0 & x \in \omega_2 \end{cases}$$

where:

- ω is the **weight vector**
- ω_0 is the **threshold weight** or **bias**
- $\mathbf{x} = [x_1, \ldots, x_n], \mathbf{w} = [w_1, \ldots, w_n]$

For a problem with 2 classes, the goal is to create a **linear function** g(x) that separates the examples belonging to the two classes.

Note: $g(x) = 0 \Rightarrow$ either class



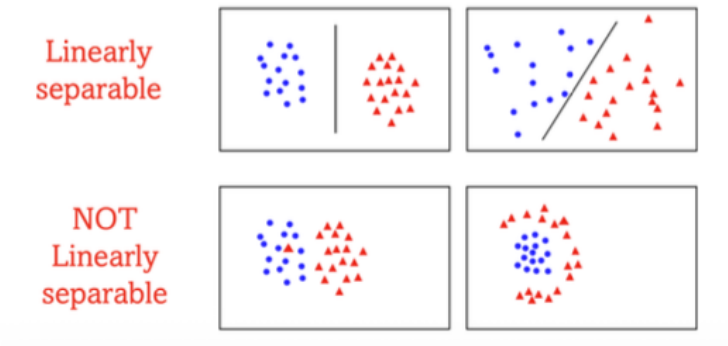
28.1 Larning the Linear Discriminant Function

Suppose we have a set of m samples $\mathbf{x}_1, \ldots, \mathbf{x}_m$ some tagged ω_1 and others tagged ω_2.

We want to use these samples to determine the weights $\mathbf{w}$ and ω_0 of the linear discriminating function.

Suppose also that there is a solution for which the probability of error is very low. Thus, a resonable approach is to look for a weight vector that classifies all of the samples correctly or that the probability of making a mistake is zero.

If such a **weight vector** exists, the samples are said to be **linear separable**

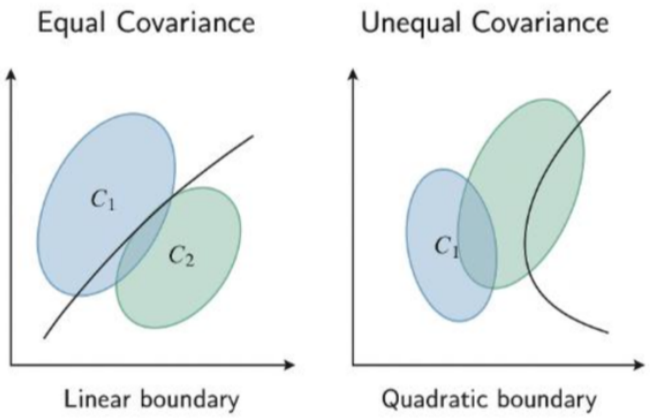


28.2 Linear Separability

Data are **linearly separable** if there exists a linear boundary (hyperplane) that perfectly separates the two classes.

Examples:

- **Linearly separable**: Classes that can be separated by a straight line (in 2D) or hyperplane (in higher dimensione)
- **NOT Linearly separable**: Classes that cannot be separated by any linear boundary

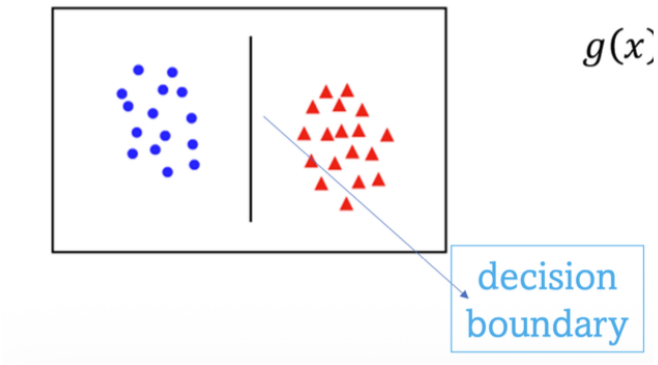


28.3 Properties of Linear Discriminant Functions

Linear Discriminant functions are:

- **Optimal for Gaussian distributions with equal covariance**: When the data follows Gaussian distributions with equal covariance matrices, the optimal decision boundary is linear
- **May not be oprimal for other data distributions**, but they are very simple to use
- **Knowledge of class densities is not required** when using linear discriminant functions
  - Therefore, we can say that this is a **non-parametric approach**

Note. With unequal covariance, the optimal decision boundary is quadratic rather than linear.



29 Linear Classifiers

Every linear discriminant functiond defines a linear classifier

But not every linear classifier is necessarily described in the form of a "discriminant function" (although in practice, the terms are often used interchangeably)

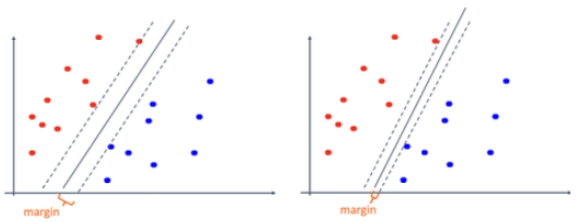
29.1 Decision Boundary

The linear discriminant function is defined as:

$$g(x) = w^T x + w_0 \Leftrightarrow \begin{cases} g(x) > 0 & x \in \omega_1 \\ g(x) < 0 & x \in \omega_2 \end{cases}$$

The **decsion boundary** is defined by $g(x) = w^T x + w_0 = 0$:

- In 2D, is a **line**
- In 3D, is a **plane**
- In nD, is a **hyperplane**



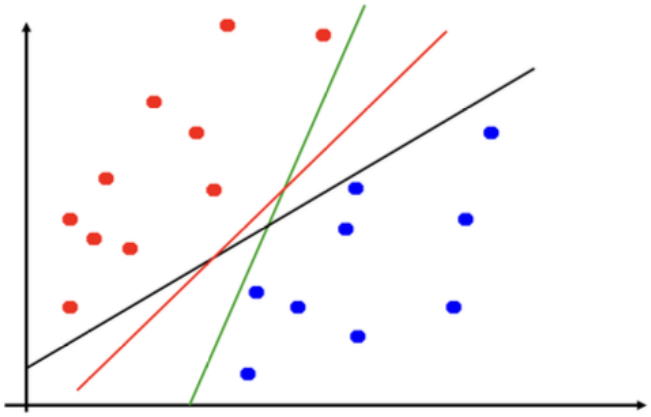
29.2 The Role of Weights

The training data is used to learn $\mathbf{w}$.

Important. Onlu $\mathbf{w}$ is needed to classify the new data point.

The equation $\mathbf{w}^T\mathbf{x}_i = 0$ defines a hyperplane passing through the origin of the feature space, haveing $\mathbf{w}$ as a normal vector.

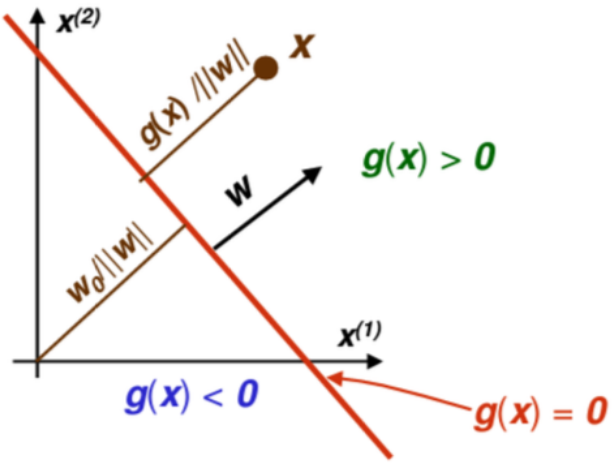
- $\mathbf{w}$ determines the orientation of the decsion boundary
- w_0 determines the location of the decision surface



29.3 Which Hyperplanes?

If the solution vector exists (if there is a hyperplane) then:

- Which hyperpplane a linear classifier choose when the data is linearly separable?
- The solutoin vector is not unique!



29.4 Optimization Approaches

There are different approaches to find the optimal weight vector:

29.4.1 Unit-Length Weight Vector

One possibility is to seek a **unit-length weight vector** that maximizes the minimum distance from the samples to the separating plane.

29.4.2 Minimum-Length Weight Vector

Another possibility is to seek the **minimum-length weight vecotor** satisfying:

$$\mathbf{w}^T \mathbf{x}_i \geq b, \quad \forall i$$

where b is a positive constant called the **margin**.

These techniques attempt to find a solution vector closer to the "middle" of the solution region:

- In this way, the resulting solution is more likely to classify new test samples correctly

29.5 The Margin

Definizione 29.1 (Margin)

The **margin** of a classifier is the distance to the closest points of either class.

A larger margin generally leads to better generalization perfromance on unseen data.

30 Multi-Class Problem

In case of a multi-class (C-class) problem:

Definizione 30.1 (Mulit-Class Classification Strategies)

One-vs-Rest Classifier

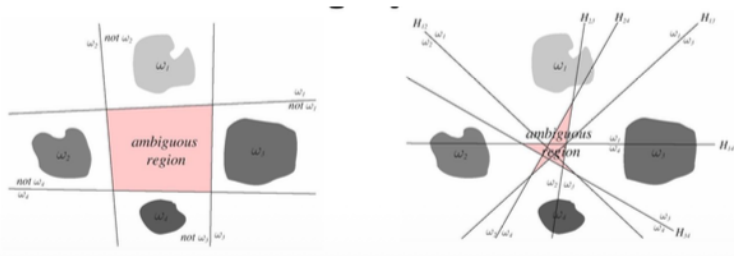
C-1 classifier $g_i(x)$ are constructed, one for each class C_i versus non-C_i.

One-vs-One Classifiers

$C(C-1)/2$ binary classifiers (one-vs-one), and then classify a test sample following a **majority voting**

Ambiguity Regions

Both approaches lead to areas of **ambiguity**.



31 LDF: Augmented Feature Vector

LDF: $g(x) = w^T x + w_0$

Definizione 31.1 (Augmented Feature Vector Transformation)

We can write it as:

$$g(\mathbf{x}) = [w_0 \quad w^T] \begin{pmatrix} 1 \\ \mathbf{x} \end{pmatrix} = \mathbf{a}^T \mathbf{y} = g(\mathbf{y})$$

where:

- $\mathbf{a}$ is the **new weight vector**
- $\mathbf{y}$ is the **new feature vector** (called the **augmented feature vector**)

We added a dummy dimension to get a completely equivalent new **homogeneous** problem.
Old problem: $g(\mathbf{x}) = w^t \mathbf{x} + w_0$, with $\mathbf{x} = [x_1, \ldots, x_d]$
New problem: $g(\mathbf{y}) = \mathbf{a}^T \mathbf{y}$ with $\mathbf{y} = [1, x_1, \ldots, x_d]$

Feature augmenting is done only for simpler notation.

Note. Throughout these lecture notes we replace $\mathbf{a}$ with $\mathbf{w}$ for simplicity, since they both refer to weights.

32 LDF: Training Error

For the rest of the lecture, assumes we have 2 classes.

We will use samples (training data) to determine weights $\mathbf{a}$ (or indicated with $\mathbf{w}$) in the discriminant function $g(\mathbf{y}) = \mathbf{a}^T\mathbf{y}$.

32.1 Criterion for Determining Weights

What should be our criterion for determining $\mathbf{a}$?
!!DA FARE DA 102 A 106

33 Perceptron

Let's learn a **binary classifier** with linear discriminant function.

Teorema 33.1 (Important note)

From this point to the rest, $\mathbf{w}$ and $\mathbf{x}$ are used for **augmented feature vector** for semplicity, instead of using $\mathbf{a}$ and $\mathbf{y}$.

To do so, we need to choose a $J(\mathbf{w})$ such taht the gradinet descent can solve it, in this way we can find the miimum of $J(\mathbf{w})$.

33.1 Perceptron Criterion Function

Definizione 33.2 (Perceptron Criterion function)

A good choiche is the **Perceptron criterion function**:

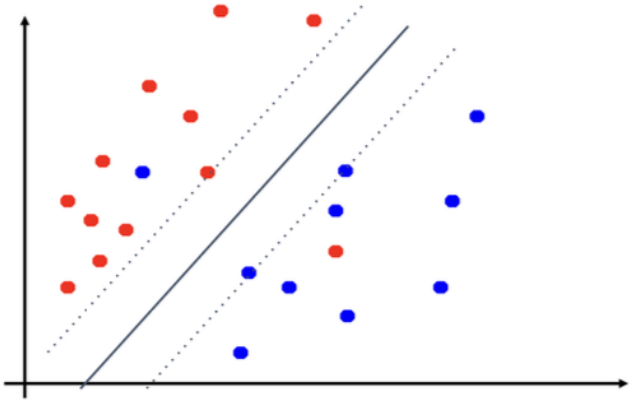
$$J(\mathbf{w}) = -\sum_{i \in X} \mathbf{w}^T\mathbf{x}_i$$

where X is a set of **samples misclassified** by $\mathbf{w}$.
Key property: $J(\mathbf{w})$ is non-negative since $\mathbf{w}^T\mathbf{x}_i < 0$ for all misclassified samples.

Important: Do not forget to "normalize" the training set by replacing all examples from class ω_2 with their negatives.

33.2 Convergence Limitation

We want to use gradient descent to solve the $J(\mathbf{w})$ so that it can converge, however, for such data (not linearly separable), we cannot converge.



33.3 Perceptron Gradient Descent

To find the minimum of $J(\mathbf{x})$, we use gradient descent, which is defined by:

- i-th component of the gradient of $J(\mathbf{x})$ is equal to $\partial J/\partial w_i$, thus:

$$J(\mathbf{w}) = -\sum_{i \in X} \mathbf{w}^T \mathbf{x}_i \quad \Rightarrow \quad \nabla J = -\sum_{i \in X} \mathbf{x}_i$$

Note that this does not depend on $\mathbf{w}$.

Teorema 33.3 (Perceptron Rule)

The update rule becomes:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + p_k \cdot \sum_{i \in X} \mathbf{x}_i, \qquad \text{when } p_k = \frac{1}{k}$$

Historical note: Perceptron is the first **Neural Network**, which was then evolved to more complex versions such as **Multilayer Perceptron**.

33.4 Perceptron Example (Non-Separable Scenario)

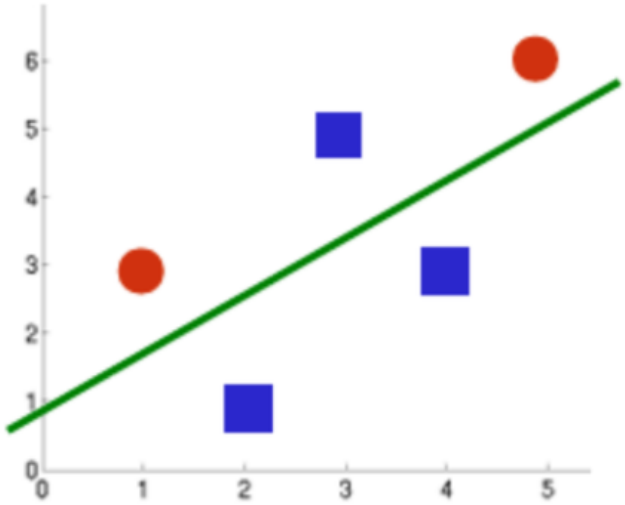
Suppose we have 2 features and 5 samples for 2 classes:

- Class 1: [2,1], [4,3], [3,5]
- Class 2: [1,3], [5,6]

These samples are not separable with a line, stille we wil try to get approximate sepration by a line.

Obtain the augmented features and normalize thte data, by doing so we obtain:

$$\begin{bmatrix}1\\2\\1\end{bmatrix}, \begin{bmatrix}1\\4\\3\end{bmatrix}, \begin{bmatrix}1\\3\\5\end{bmatrix}, \begin{bmatrix}-1\\-1\\-3\end{bmatrix}, \begin{bmatrix}-1\\-5\\-6\end{bmatrix}$$



33.5 Applying the Perceptron Method

Let's apply the perceptron method:

- Initial weights $[1 \quad 1 \quad 1]$
- This is a line $x_1 + x_2 + 1 = 0$
- Fixed learning rate $p(k) = 1$

Iteration 1:

$$\mathbf{w}_1^T \mathbf{x}_1 \Rightarrow [1\,1\,1][1\,2\,1]^T > 0$$
$$\mathbf{w}_1^T \mathbf{x}_2 \Rightarrow [1\,1\,1][1\,4\,3]^T > 0$$
$$\mathbf{w}_1^T \mathbf{x}_3 \Rightarrow [1\,1\,1][1\,3\,5]^T > 0$$

$$\mathbf{w}_1^T \mathbf{x}_4 \Rightarrow [1\,1\,1][-1\ -1\ -3]^T = -5 < 0$$
Update w!!! $\mathbf{w}_2 = [1\,1\,1] + [-1\ -1\ -3] = [0\,0\,2]$

Continue iteration:

$$\mathbf{w}_2^T \mathbf{x}_5 \Rightarrow [0\,0\,-2][-1\,-5\,-6]^T > 0$$

Iteration 2:

$$\mathbf{w}_2^T \mathbf{x}_1 \Rightarrow [0\,0\,-2][1\,2\,1]^T < 0$$

Update w again!!!

$$\mathbf{w}_3 = [0\,0\,-2] + [1\,2\,1] = [1\,2\,-1]$$

$$\dots$$

$$\dots$$

Definizione 33.4 (Non-Convergence Result)

We can continue this forever since there is no solution vector $\mathbf{w}$ satisfying the perceptron rule.
The data is **not linearly separable**, so the perceptron algorithm will not converge to a solution.

33.6 Need for Relaxation

We want to learn something like this, but our Perceptron constraints does not allow!

Solution: Let's apply some **relaxation**.

33.7 Perceptron Summary

Definizione 33.5 (Perceptron Convergence Properties)

- If classes are **linearaly separable**, the perceptron rule is **guaranteed to converge** to a valid solution
  - Some versions of the perceptron rule use a **variable learning rate** $p(k)$; for now, we fixed it as $\frac{1}{k}$.
- However, if the two classes are **not linearly separable**, the perceptron rule will **not converge**
  - Since no weight vector $\mathbf{w}$ can correctly classify every sample in a non-serparable dataset, the corrections in the perceptron rule will never cease
  - One ad-hoc solution to this problem is to enforce convergence by using variable learning rates $p(k)$ taht approach zero as $k \to \infty$
  - We will see once we introduce NN architectures